

modelduel

Dos modelos de IA, una tarea, los mismos tests. Un informe con quién acierta, cuánto tarda y cuánto cuesta.

Repositorio	https://github.com/BertMarti/modelduel
Web publicada	https://bertmarti.github.io/modelduel/
Stack	Python 3.12 (solo biblioteca estándar) · pytest · ruff · GitHub Actions · GitHub Pages
Fecha	30 de septiembre de 2026
Autoría	Alberto Martínez, con un equipo de agentes de IA (Claude Code + OpenCode)

Contenido

1. Qué es y para quién
 2. Funcionamiento a nivel de usuario
 3. Funcionamiento a nivel técnico
 4. Flujo de un duelo
 5. Funcionalidades
 6. Calidad, CI/CD y equipo
 7. Cómo explotarlo económicamente
 8. Sigüientes pasos y acciones necesarias por ti, Alberto
- Anexo: código fuente

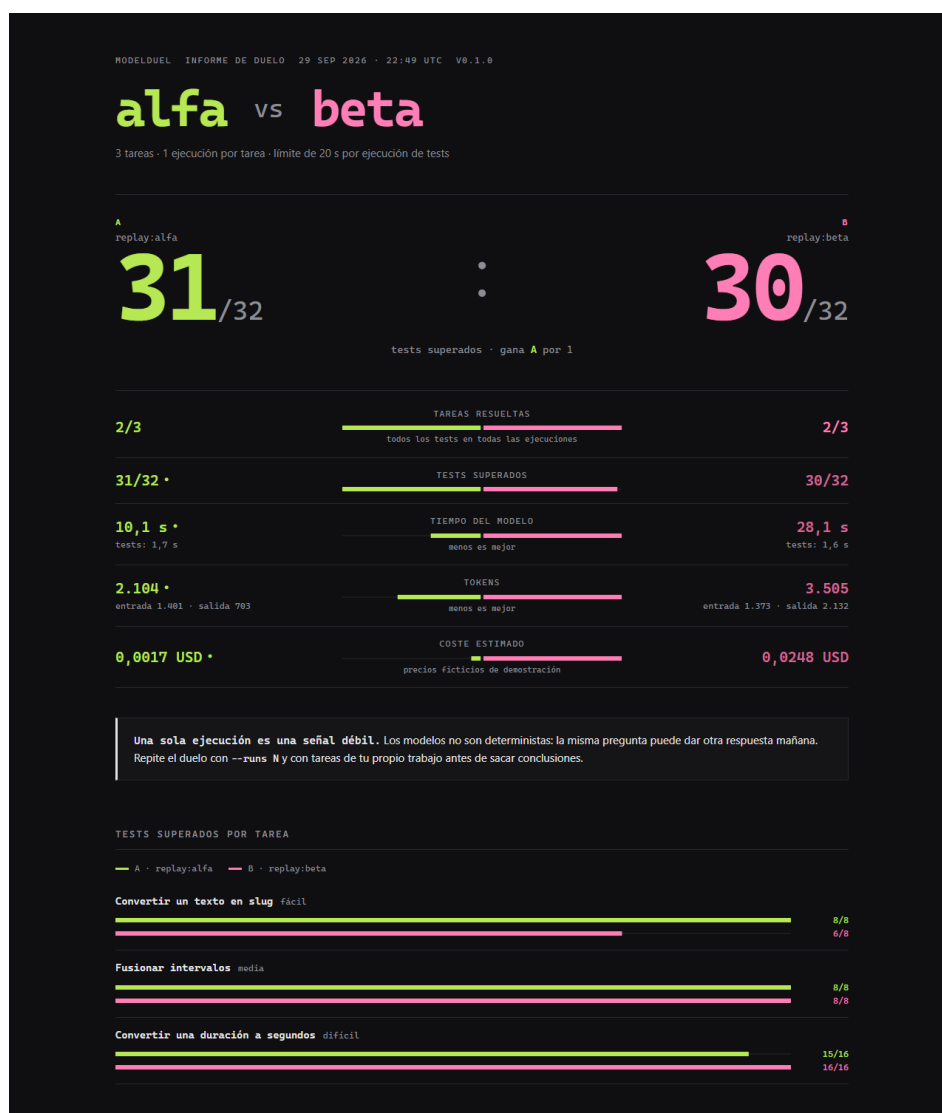
Qué es y para quién

modelduel es una herramienta de línea de comandos que enfrenta a dos modelos de IA con el mismo problema de programación. Les pide el código, ejecuta sobre cada respuesta los mismos tests y genera un informe HTML con el marcador: tests superados, tareas resueltas, tiempo, tokens y coste estimado.

Convierte en herramienta el ejercicio «tu propia comparativa» de la *Guía de modelos de IA 2026* del curso: los rankings miden tareas que no son las tuyas; la forma honesta de elegir es probar con tu material y tus tests.

Perfil	Para qué lo usa
Estudiante o desarrollador	Decidir qué modelo usar en su día a día con datos, no con intuiciones.
Equipo técnico	Validar un cambio de proveedor o de modelo antes de adoptarlo.
Docente	Preparar ejercicios con tests y comparar cómo los resuelven distintos modelos.
Creador de contenido	Publicar duelos reproducibles con informe descargable.

Funcionamiento a nivel de usuario



Informe de demostración: marcador, métricas enfrentadas y tests superados por tarea.

Probar la demo sin instalar nada

Abre <https://bertmarti.github.io/modelduel/demo/>. Es un informe real generado en el CI con dos contendientes ficticios (*alfa* y *beta*) cuyas respuestas están grabadas: no hace falta ninguna clave ni cuesta nada.

Usarlo en tu ordenador

1. Instala Python 3.12 o superior.
2. Instala la herramienta y pytest: `pip install git+https://github.com/BertMarti/modelduel`
`pytest`
3. Prueba la demo en local: `modelduel run examples/tasks --a replay:alfa --b replay:beta --out runs/demo` y abre `runs/demo/index.html`.
4. Para un duelo real, define tus claves como variables de entorno (por ejemplo `GEMINI_API_KEY` y `OPENAI_API_KEY`).
5. Lanza el duelo: `modelduel run examples/tasks --a gemini:<modelo> --b openai:<modelo> --runs 3 --out runs/duelo`
6. Abre el informe `runs/duelo/index.html`. Puedes imprimirlo o guardarlo como PDF desde el navegador.

Crear tus propias tareas

Una tarea es una carpeta con tres archivos: `task.md` (el enunciado que se envía al modelo, pidiendo una función concreta con nombre y firma), `test_task.py` (tests pytest que importan desde `solution`) y, opcionalmente, `meta.toml` (título, dificultad y orden).

```
examples/tasks/slugify/
├─ task.md          # enunciado para el modelo
├─ test_task.py     # tests pytest: from solution import slugify
└─ meta.toml        # title, difficulty, order
```

Leer el informe

- **Marcador:** tests superados y tareas resueltas por cada contendiente (A en lima, B en rosa).
- **Métricas enfrentadas:** tiempo del modelo, tokens de entrada y salida, coste estimado.
- **Barras por tarea** y tabla de resultados.
- **Detalle desplegable** por tarea con el código de cada modelo y la salida de los tests.
- **Aviso:** «Una sola ejecución es una señal débil». Usa `--runs` para repetir.

Seguridad. El código que escriben los modelos se ejecuta en tu máquina. `modelduel` lo aísla en un directorio temporal, en un subproceso con límite de tiempo y sin tus variables de entorno secretas, pero no es un aislamiento real. Úsalo con tareas de confianza e, idealmente, en un contenedor o una máquina virtual.

Funcionamiento a nivel técnico

Arquitectura

Módulo	Responsabilidad
cli.py	Punto de entrada con argparse: comandos run, report y list-tasks; resumen tipo marcador en consola.
tasks.py	Descubre y carga tareas (task.md, test_task.py, meta.toml con tomllib).
providers/	Interfaz común complete(prompt, task_id) → texto, tokens y latencia. Implementaciones: replay, gemini y openai (compatible).
extract.py	Extrae el bloque de código de la respuesta: ```python, ```py, ~~~, bloques sin lenguaje o sin cerrar.
runner.py	Escribe solution.py y los tests en un directorio temporal, ejecuta pytest en subprocesso con timeout y lee el XML JUnit.
pricing.py	Coste = entrada/1e6 × tarifa entrada + salida/1e6 × tarifa salida. Sin precio: «sin datos», nunca inventado.
duel.py / results.py	Orquesta el duelo, agrega varias ejecuciones y serializa results.json.
report/	Informe HTML autocontenido con string.Template, CSS en línea y gráficas SVG generadas en Python; sin JavaScript.

Decisiones de diseño

- **Solo biblioteca estándar en ejecución** (urllib, argparse, tomllib, xml): instalar es trivial y no hay dependencias que auditar.
- **Proveedor replay** con respuestas grabadas: la demo, los tests y el CI funcionan sin claves ni coste.
- **Resultados por XML JUnit** en vez de leer la salida de texto de pytest: más robusto entre versiones y sistemas.
- **Claves solo por variables de entorno**; Gemini recibe la clave en una cabecera, no en la URL; nunca aparecen en errores ni en el informe; no llegan al subprocesso.
- **Informe sin JavaScript**: se abre en cualquier sitio, se puede adjuntar o imprimir y no ejecuta nada.

Flujo de un duelo



Funcionalidades

Funcionalidad	Detalle
Comando run	Duelo entre --a y --b sobre una tarea o una carpeta; --runs, --timeout, --prices, --replays, --out.
Comando report	Regenera el HTML a partir de results.json.
Comando list-tasks	Lista las tareas de una carpeta con título y dificultad.
Proveedor replay	Respuestas grabadas con front-matter de tokens y latencia.
Proveedor gemini	API generateContent de Google; GEMINI_API_KEY.
Proveedor openai	Cualquier API compatible (OpenAI, OpenRouter, Ollama); OPENAI_BASE_URL y OPENAI_API_KEY.
Ejecución segura	Directorio temporal, subprocesso con timeout, sin stdin y sin variables secretas.
Coste estimado	Tabla de precios ampliable con --prices; sin precios reales incrustados.
Informe HTML	Marcador, métricas, barras por tarea, tabla, detalle con código y salida, hoja de impresión.
Tres tareas originales	slugify, merge_intervals y parse_duration, de dificultad creciente.
Web y demo	site/ con la explicación y el informe de demostración generado en el CI.

Calidad, CI/CD y equipo

- **Tests:** pytest sin red: extracción, runner (soluciones correctas, erróneas, que no importan, que se cuelgan), proveedores con respuestas simuladas, costes, informe (escapado de <script>) y CLI de extremo a extremo.
- **CI** (ci.yml): en cada PR y en main, en Ubuntu y Windows: ruff, formato, pytest y generación de la demo.
- **CD** (deploy.yml): al actualizar main, genera la demo y publica site/ en GitHub Pages.
- **Git:** main protegida (solo por PR), commits convencionales, una rama por agente: agent/builder, agent/qa y agent/opencode-docs.
- **Contexto para IA:** AGENTS.md (reglas, stack, diseño), CLAUDE.md y opencode.json (cargan AGENTS.md y MEMORY.md) y MEMORY.md (estado vivo, actualizado por cada agente).
- **Revisión QA** (Claude Opus, 132 tests, cobertura 94 % con mínimo del 90 % en el CI): procesos nietos que colgaban el runner en Windows (ahora se mata el árbol con un Job Object o un grupo de procesos), salida enorme acotada, tareas con BOM, tests rotos que culpaban a los modelos, cortes de red, HTML sin escapar en el informe, marcador con el total equivocado, contraste insuficiente y CLI en español con códigos de salida (0, 1, 2 y 130).
- **Documentación** (Claude Sonnet): docs/USO.md con instalación por sistema, duelos con Gemini, OpenAI, OpenRouter y Ollama, una tarea nueva paso a paso (es_palindromo), FAQ y solución de problemas; CONTRIBUTING.md.
- **Pull requests:** #1 builder, #2 qa (sobre builder) y #3 docs (sobre qa), todos con el CI en verde.

Cómo explotarlo económicamente

La herramienta es gratuita y de código abierto; el valor está en los datos, la comodidad y la audiencia. Propuestas ordenadas de menor a mayor esfuerzo:

Idea	Cómo	Esfuerzo
Contenido y audiencia	Publicar duelos mensuales (artículo, vídeo corto, newsletter) con el informe como pieza visual. Monetiza con patrocinios.	Bajo
Packs de tareas	Colecciones de tareas con tests de calidad por dominio (datos, web, algoritmos, entrevistas). Gratis las básicas, de pago las avanzadas.	Bajo
GitHub Sponsors	Patrocinio para mantener proveedores y tareas.	Bajo
Acción de CI	Una GitHub Action que ejecuta el duelo en cada PR y comenta el marcador: útil para equipos que evalúan modelos.	Medio
Servicio alojado	Versión web que ejecuta los duelos en contenedores aislados, con historial y comparativas. Suscripción para equipos.	Alto
Consultoría	Evaluaciones a medida para empresas que eligen proveedor de IA.	Medio

Antes de invertir en cualquiera: mide si hay interés (estrellas, visitas a la demo, descargas) y pregunta a 5-10 posibles usuarios qué pagarían.

Siguientes pasos y acciones necesarias por ti, Alberto

Acciones necesarias por ti

1. **Fusionar los PR en orden** (builder → qa → docs). El sistema de permisos no me deja fusionar en main. Al fusionar se despliega la web automáticamente.
2. Comprobar la web y la demo publicadas: <https://bertmarti.github.io/modelduel/> y [/demo/](https://bertmarti.github.io/demo/).
3. Probar un duelo real con tus claves en variables de entorno (las APIs reales de Gemini y OpenAI no se han llamado nunca: solo están probadas con respuestas simuladas).
4. Si ChatGPT es de tu empresa, confirma que puedes usarlo para esto.

Mejoras técnicas propuestas

- Ejecutar el código dentro de un contenedor (Docker) cuando esté disponible.
- Más de dos contendientes (liga) y comparación histórica entre duelos.
- Publicar el paquete en PyPI para instalarlo con `pip install modelduel`.
- Medir también la calidad del código (lint, complejidad) además de los tests.

Anexo: código fuente

Archivos principales tal y como están en main al generar este documento. Las líneas muy largas se parten y continúan en la línea siguiente.

pyproject.toml · 58 líneas

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "modelduel"
version = "0.1.0"
description = "Dos modelos, una tarea, los mismos tests: compara modelos de IA programando."
readme = "README.md"
requires-python = ">=3.12"
license = "MIT"
authors = [{ name = "Alberto Martínez" }]
keywords = ["llm", "benchmark", "evaluation", "cli"]
classifiers = [
    "Programming Language :: Python :: 3",
    "Programming Language :: Python :: 3.12",
    "License :: OSI Approved :: MIT License",
    "Environment :: Console",
]
dependencies = []

[project.optional-dependencies]
dev = ["pytest>=8", "pytest-cov>=5", "ruff>=0.6"]

[project.scripts]
modelduel = "modelduel.cli:main"

[project.urls]
Homepage = "https://bertmarti.github.io/modelduel/"
Source = "https://github.com/BertMarti/modelduel"

[tool.hatch.build.targets.wheel]
packages = ["src/modelduel"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q"

[tool.ruff]
line-length = 100
target-version = "py312"
extend-exclude = ["examples/replays", "site"]

[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "SIM"]

[tool.ruff.lint.per-file-ignores]
"examples/tasks/*/test_task.py" = ["I001"]

[tool.coverage.run]
source = ["modelduel"]
branch = true

[tool.coverage.report]
show_missing = true
skip_covered = true
fail_under = 90
```

src/modelduel/cli.py · 262 líneas

```
"""Interfaz de línea de órdenes de modelduel."""

from __future__ import annotations

import argparse
import math
import re
import sys
from pathlib import Path

from modelduel import __version__
from modelduel.duel import run_duel
from modelduel.pricing import PricingError, load_prices
from modelduel.providers import ProviderError, get_provider
from modelduel.report import write_report
from modelduel.report.html import fmt_cost, fmt_int, fmt_seconds, plural
from modelduel.results import ResultsError, load_results, save_results
from modelduel.runner import DEFAULT_TIMEOUT, RunnerError, ensure_pytest_available
from modelduel.tasks import TaskError, discover_tasks, is_task_dir

# Códigos de salida: 0 duelo completado (aunque los modelos fallen tests), 1 no se pudieron
# escribir los resultados, 2 error de uso o de configuración, 130 interrumpido con Ctrl+C.
EXIT_OK = 0
EXIT_ERROR = 1
EXIT_USAGE = 2
EXIT_INTERRUPTED = 130

class OutputError(Exception):
    """No se pudo escribir en la carpeta de salida."""

# argparse no trae traducciones: se traducen sus mensajes de error más habituales.
_ARGPARSE_ES = [
    (r"the following arguments are required: (.+)", r"faltan argumentos obligatorios: \1"),
    (
        r"argument orden: invalid choice: (.?) \((choose from (.+)\))",
        r"orden no válida: \1 (elige entre \2)",
    ),
    (
        r"argument (.?): invalid choice: (.?) \((choose from (.+)\))",
        r"argumento \1: valor no válido: \2 (elige entre \3)",
    ),
    (r"argument (.?): invalid \w+ value: (.+)", r"argumento \1: valor no válido: \2"),
    (r"argument (.?): expected one argument", r"argumento \1: necesita un valor"),
    (r"unrecognized arguments: (.+)", r"argumentos no reconocidos: \1"),
    (r"ambiguous option: (.?) could match (.+)", r"opción ambigua: \1 puede ser \2"),
]

def translate_argparse(message: str) -> str:
    for pattern, replacement in _ARGPARSE_ES:
        translated, n = re.subn(f"^{pattern}$", replacement, message)
        if n:
            return translated
    return message

class _SpanishHelpFormatter(argparse.HelpFormatter):
    def add_usage(self, usage, actions, groups, prefix=None):
        super().add_usage(usage, actions, groups, prefix="uso: " if prefix is None else prefix)

class SpanishArgumentParser(argparse.ArgumentParser):
    """ArgumentParser con ayuda y errores en español y código de salida ``EXIT_USAGE``."""

    def __init__(self, *args, **kwargs) -> None:
        kwargs["add_help"] = False
        kwargs.setdefault("formatter_class", _SpanishHelpFormatter)
        super().__init__(*args, **kwargs)
        self._positionals.title = "argumentos posicionales"
        self._optionals.title = "opciones"
        self.add_argument("-h", "--help", action="help", help="muestra esta ayuda y sale")

    def error(self, message: str): # type: ignore[override]
        self.print_usage(sys.stderr)
```

```

        self.exit(EXIT_USAGE, f"{self.prog}: error: {translate_argparse(message)}\n")

def build_parser() -> argparse.ArgumentParser:
    parser = SpanishArgumentParser(
        prog="modelduel",
        description="Dos modelos, una tarea, los mismos tests.",
        epilog="Aviso: el código generado por los modelos se ejecuta en tu máquina.",
    )
    parser.add_argument(
        "--version",
        action="version",
        version=f"modelduel {__version__}",
        help="muestra la versión y sale",
    )
    sub = parser.add_subparsers(dest="command", required=True, metavar="orden")

    run = sub.add_parser("run", help="enfrenta a dos modelos y genera el informe")
    run.add_argument("tasks", type=Path, help="carpeta de tareas o carpeta de una tarea")
    run.add_argument("--a", required=True, metavar="PROVEEDOR:MODELO", help="contendiente A")
    run.add_argument("--b", required=True, metavar="PROVEEDOR:MODELO", help="contendiente B")
    run.add_argument("--runs", type=int, default=1, metavar="N", help="ejecuciones por tarea (1)")
    run.add_argument(
        "--timeout",
        type=float,
        default=DEFAULT_TIMEOUT,
        metavar="S",
        help=f"límite en segundos para los tests de cada respuesta ({DEFAULT_TIMEOUT:g})",
    )
    run.add_argument("--prices", type=Path, metavar="F.json", help="tabla de precios adicional")
    run.add_argument(
        "--replays", type=Path, metavar="DIR", help="carpeta de respuestas grabadas para replay"
    )
    run.add_argument("--out", type=Path, required=True, metavar="DIR", help="carpeta de salida")

    report = sub.add_parser("report", help="regenera el informe HTML desde results.json")
    report.add_argument("results", type=Path, help="ruta a results.json")
    report.add_argument("--out", type=Path, required=True, metavar="DIR", help="carpeta de salida")

    lst = sub.add_parser("list-tasks", help="lista las tareas de una carpeta")
    lst.add_argument("tasks", type=Path, help="carpeta de tareas")
    return parser

def main(argv: list[str] | None = None) -> int:
    for stream in (sys.stdout, sys.stderr):
        if hasattr(stream, "reconfigure"):
            stream.reconfigure(encoding="utf-8", errors="replace")
    args = build_parser().parse_args(argv)
    try:
        if args.command == "run":
            return cmd_run(args)
        if args.command == "report":
            return cmd_report(args)
        return cmd_list_tasks(args)
    except (TaskError, ProviderError, PricingError, ResultsError, RunnerError) as exc:
        print(f"modelduel: error: {exc}", file=sys.stderr)
        return EXIT_USAGE
    except OutputError as exc:
        print(f"modelduel: error: {exc}", file=sys.stderr)
        return EXIT_ERROR
    except KeyboardInterrupt:
        print("\nmodelduel: interrumpido.", file=sys.stderr)
        return EXIT_INTERRUPTED

def cmd_run(args: argparse.Namespace) -> int:
    if args.runs < 1:
        raise TaskError("--runs debe ser 1 o más.")
    if not math.isfinite(args.timeout) or args.timeout <= 0:
        raise TaskError("--timeout debe ser un número de segundos mayor que 0.")
    tasks = discover_tasks(args.tasks)
    prices = load_prices(args.prices)
    # examples/tasks[<tarea>] -> examples/replays
    tasks_root = args.tasks.parent if is_task_dir(args.tasks) else args.tasks
    replay_dirs = [
        d

```

```

        for d in (args.replays, tasks_root.parent / "replays", Path("examples/replays"))
        if d is not None
    ]
    providers = {"a": get_provider(args.a, replay_dirs), "b": get_provider(args.b, replay_dirs)}
    ensure_pytest_available()
    # Antes de gastar llamadas a las APIs: la carpeta de salida tiene que poder crearse.
    _prepare_out(args.out)

    print(
        f"modelduel {__version__} · {plural(len(tasks), 'tarea', 'tareas')} · "
        f"{plural(args.runs, 'ejecución', 'ejecuciones')} por tarea"
    )
    print(f"  A {providers['a'].spec}")
    print(f"  B {providers['b'].spec}")
    print("  Aviso: el código de los modelos se ejecuta en esta máquina (temporal + límite).")
    print()
    results = run_duel(
        tasks,
        providers,
        prices,
        runs=args.runs,
        timeout=args.timeout,
        progress=lambda line: print(line, flush=True),
    )
    html_path = _write_outputs(results, args.out, with_json=True)
    print()
    print(scoreboard(results))
    print()
    print(f"  Resultados {args.out / 'results.json'}")
    print(f"  Informe {html_path}")
    return 0

def cmd_report(args: argparse.Namespace) -> int:
    results = load_results(args.results)
    _prepare_out(args.out)
    try:
        html_path = _write_outputs(results, args.out, with_json=False)
    except (KeyError, TypeError, ValueError, AttributeError) as exc:
        raise ResultsError(
            f"{args.results} no parece un results.json de modelduel ({type(exc).__name__}: {exc})."
        ) from exc
    print(f"Informe regenerado: {html_path}")
    return 0

def _prepare_out(out: Path) -> None:
    try:
        out.mkdir(parents=True, exist_ok=True)
    except OSError as exc:
        raise OutputError(f"no se pudo escribir en {out}: {exc.strerror or exc}.") from exc

def _write_outputs(results: dict, out: Path, with_json: bool) -> Path:
    try:
        if with_json:
            save_results(results, out / "results.json")
            return write_report(results, out)
    except OSError as exc:
        raise OutputError(f"no se pudo escribir en {out}: {exc.strerror or exc}.") from exc

def cmd_list_tasks(args: argparse.Namespace) -> int:
    tasks = discover_tasks(args.tasks)
    width = max(len(t.id) for t in tasks)
    for task in tasks:
        extra = f" [{task.difficulty}]" if task.difficulty else ""
        print(f"{task.id:<{width}} {task.expected_tests:>2} tests {task.title}{extra}")
    return 0

def scoreboard(results: dict) -> str:
    s = results["summary"]
    a, b = s["a"], s["b"]
    specs = (results["contenders"]["a"]["spec"], results["contenders"]["b"]["spec"])
    rows = [
        ("", f"A {specs[0]}", f"B {specs[1]}"),

```

```

(
    "Tests superados",
    f"{a['tests_passed']}/{a['tests_total']}",
    f"{b['tests_passed']}/{b['tests_total']}",
),
(
    "Tareas resueltas",
    f"{a['tasks_solved']}/{a['tasks_total']}",
    f"{b['tasks_solved']}/{b['tasks_total']}",
),
("Tiempo del modelo", fmt_seconds(a["latency_s"]), fmt_seconds(b["latency_s"])),
(
    "Tokens entrada/salida",
    f"{fmt_int(a['input_tokens'])}/{fmt_int(a['output_tokens'])}",
    f"{fmt_int(b['input_tokens'])}/{fmt_int(b['output_tokens'])}",
),
("Coste estimado", fmt_cost(a["cost"], a["currency"]), fmt_cost(b["cost"], b["currency"])),
]
w0 = max(len(r[0]) for r in rows)
w1 = max(len(r[1]) for r in rows)
w2 = max(len(r[2]) for r in rows)
lines = [f" {r[0]:<{w0}} {r[1]:>{w1}} {r[2]:>{w2}}" for r in rows]
rule = " " + "-" * (w0 + w1 + w2 + 6)
lines.insert(1, rule)
if a["fictitious_price"] or b["fictitious_price"]:
    lines.append(" (precios ficticios de demostración)")
return "\n".join(lines)

if __name__ == "__main__":
    raise SystemExit(main())

```

src/modelduel/duel.py · 119 líneas

```
"""Orquesta el duelo: pide el código a cada modelo y ejecuta los tests de cada tarea."""

from __future__ import annotations

from collections.abc import Callable
from datetime import UTC, datetime

from modelduel import __version__
from modelduel.extract import extract_code
from modelduel.pricing import Price, compute_cost, find_price
from modelduel.providers import Provider, ProviderError
from modelduel.results import SCHEMA_VERSION, SIDES, summarize
from modelduel.runner import DEFAULT_TIMEOUT, TestRun, build_prompt, run_tests
from modelduel.tasks import Task

Progress = Callable[[str], None]

def run_duel(
    tasks: list[Task],
    providers: dict[str, Provider],
    prices: dict[str, Price],
    runs: int = 1,
    timeout: float = DEFAULT_TIMEOUT,
    progress: Progress | None = None,
) -> dict:
    progress = progress or (lambda _msg: None)
    contender_prices = {side: find_price(providers[side].spec, prices) for side in SIDES}
    results: dict = {
        "schema": SCHEMA_VERSION,
        "tool": "modelduel",
        "version": __version__,
        "created_at": datetime.now(UTC).isoformat(timespec="seconds"),
        "runs": runs,
        "timeout_s": timeout,
        "contenders": {
            side: {
                "spec": providers[side].spec,
                "price": contender_prices[side].to_dict() if contender_prices[side] else None,
            }
            for side in SIDES
        },
        "tasks": [],
    }
    for task in tasks:
        prompt = build_prompt(task)
        entry = {
            "id": task.id,
            "title": task.title,
            "difficulty": task.difficulty,
            "statement": task.statement,
            "tests_expected": task.expected_tests,
            "results": {side: [] for side in SIDES},
        }
        for run in range(1, runs + 1):
            for side in SIDES:
                attempt = run_attempt(
                    task, prompt, providers[side], contender_prices[side], timeout
                )
                attempt["run"] = run
                entry["results"][side].append(attempt)
                progress(_progress_line(task, side, run, runs, attempt))
            results["tasks"].append(entry)
        results["summary"] = summarize(results)
    return results

def run_attempt(
    task: Task, prompt: str, provider: Provider, price: Price | None, timeout: float
) -> dict:
    try:
        response = provider.complete(prompt, task_id=task.id)
    except Exception as exc: # noqa: BLE001 - un intento fallido no debe tumbar todo el duelo
        message = str(exc) if isinstance(exc, ProviderError) else _unexpected(exc)
        test_run = TestRun(status="provider_error", total=task.expected_tests, message=message)
        attempt = test_run.to_dict()
```

```

        attempt.update(
            latency_s=None,
            input_tokens=None,
            output_tokens=None,
            cost=None,
            code=None,
            response="",
        )
    return attempt
text = clean_text(response.text)
code = extract_code(text)
test_run = run_tests(code, task, timeout=timeout)
attempt = test_run.to_dict()
attempt.update(
    latency_s=round(response.latency_s, 3),
    input_tokens=response.input_tokens,
    output_tokens=response.output_tokens,
    cost=compute_cost(price, response.input_tokens, response.output_tokens),
    code=code,
    response=text if code is None else "",
)
return attempt

def clean_text(text: str | None) -> str:
    """Sustituye los sustitutos UTF-16 sueltos (válidos en JSON, no en UTF-8) por «?»."""
    if not text:
        return ""
    return text.encode("utf-8", errors="replace").decode("utf-8")

def _unexpected(exc: Exception) -> str:
    return f"Error inesperado del proveedor ({type(exc).__name__}): {exc}"

def _progress_line(task: Task, side: str, run: int, runs: int, attempt: dict) -> str:
    run_label = f" #{run}" if runs > 1 else ""
    mark = "ok" if attempt["solved"] else "--"
    detail = f"{attempt['passed']}/{attempt['total']}"
    if attempt["status"] not in ("ok",):
        detail += f" ({attempt['status']})"
    return f" {mark} {task.id + run_label:<24} {side.upper()} {detail}"

```

src/modelduel/tasks.py · 158 líneas

```
"""Carga de tareas: una carpeta con ``task.md``, ``test_task.py`` y ``meta.toml`` opcional."""

from __future__ import annotations

import ast
import tomllib
from dataclasses import dataclass, field
from pathlib import Path

TASK_FILE = "task.md"
TEST_FILE = "test_task.py"
META_FILE = "meta.toml"

class TaskError(Exception):
    """Error al cargar una tarea."""

@dataclass
class Task:
    id: str
    path: Path
    statement: str
    title: str
    difficulty: str = ""
    order: int = 1000
    meta: dict = field(default_factory=dict)

    @property
    def test_path(self) -> Path:
        return self.path / TEST_FILE

    @property
    def expected_tests(self) -> int:
        return count_tests(read_text(self.test_path))

def read_text(path: Path) -> str:
    """Lee UTF-8 aceptando el BOM que añaden algunos editores de Windows."""
    try:
        return path.read_text(encoding="utf-8-sig")
    except UnicodeDecodeError as exc:
        raise TaskError(f"{path} no está en UTF-8: {exc.reason}.") from exc

def is_task_dir(path: Path) -> bool:
    return (path / TASK_FILE).is_file() and (path / TEST_FILE).is_file()

def load_task(path: Path) -> Task:
    path = Path(path)
    if not is_task_dir(path):
        raise TaskError(f"{path} no es una tarea: necesita {TASK_FILE} y {TEST_FILE}.")
    statement = read_text(path / TASK_FILE).strip()
    test_path = path / TEST_FILE
    try:
        ast.parse(read_text(test_path))
    except SyntaxError as exc:
        # Mejor avisar ahora que atribuir el fallo a los modelos como «no se pudo importar».
        raise TaskError(
            f"{test_path} tiene un error de sintaxis en la línea {exc.lineno}: {exc.msg}."
        ) from exc
    meta: dict = {}
    meta_path = path / META_FILE
    if meta_path.is_file():
        try:
            meta = tomllib.loads(read_text(meta_path))
        except tomllib.TOMLDecodeError as exc:
            raise TaskError(f"{meta_path} no es TOML válido: {exc}") from exc
    title = str(meta.get("title") or _title_from_statement(statement) or path.name)
    return Task(
        id=path.name,
        path=path,
        statement=statement,
        title=title,
        difficulty=str(meta.get("difficulty", "")),
    )
```

```

        order=int(meta.get("order", 1000)),
        meta=meta,
    )

def discover_tasks(path: Path) -> list[Task]:
    """Devuelve la tarea de ``path`` o todas las tareas de sus subcarpetas, ordenadas."""
    path = Path(path)
    if not path.exists():
        raise TaskError(f"No existe la ruta {path}.")
    if is_task_dir(path):
        return [load_task(path)]
    if not path.is_dir():
        raise TaskError(f"{path} no es una carpeta de tareas.")
    tasks = [load_task(child) for child in sorted(path.iterdir()) if is_task_dir(child)]
    if not tasks:
        raise TaskError(f"No hay tareas en {path}.")
    return sorted(tasks, key=lambda t: (t.order, t.id))

def _title_from_statement(statement: str) -> str:
    for line in statement.splitlines():
        line = line.strip()
        if line.startswith("#"):
            return line.lstrip("#").strip()
    return ""

def count_tests(source: str) -> int:
    """Cuenta los tests de un archivo pytest sin ejecutarlo (incluye ``parametrize`` literal)."""
    try:
        tree = ast.parse(source)
    except SyntaxError:
        return 0

    # Constantes de módulo (``CASOS = [...]``) usadas en ``parametrize``.
    constants: dict[str, int] = {}
    for node in tree.body:
        if isinstance(node, ast.Assign) and isinstance(node.value, ast.List | ast.Tuple):
            for target in node.targets:
                if isinstance(target, ast.Name):
                    constants[target.id] = len(node.value.elts)

    def n_values(node: ast.expr | None) -> int | None:
        if isinstance(node, ast.List | ast.Tuple):
            return len(node.elts)
        if isinstance(node, ast.Name):
            return constants.get(node.id)
        return None

    def weight(func: ast.FunctionDef | ast.AsyncFunctionDef) -> int:
        total = 1
        for deco in func.decorator_list:
            if not (
                isinstance(deco, ast.Call)
                and isinstance(deco.func, ast.Attribute)
                and deco.func.attr == "parametrize"
            ):
                continue
            values = deco.args[1] if len(deco.args) >= 2 else None
            for kw in deco.keywords:
                if kw.arg == "argvalues":
                    values = kw.value
            n = n_values(values)
            if n is not None:
                total *= max(n, 1)
        return total

    count = 0
    for node in tree.body:
        if isinstance(node, ast.FunctionDef | ast.AsyncFunctionDef) and node.name.startswith(
            "test"
        ):
            count += weight(node)
        elif isinstance(node, ast.ClassDef) and node.name.startswith("Test"):
            for item in node.body:
                if isinstance(item, ast.FunctionDef | ast.AsyncFunctionDef) and (

```

```

        item.name.startswith("test")
    ):
        count += weight(item)
return count

```

src/modelduel/extract.py · 52 líneas

"""Extracción del bloque de código Python de la respuesta de un modelo."""

```

from __future__ import annotations

import re
import textwrap

_FENCE_RE = re.compile(
    r"^[ \t]*(?P<fence>`{3,}|~{3,})[ \t]*(?P<lang>[\w+#.-]*)[^\n]*\n"
    r"(?P<body>.*?)"
    r"(\?:^[ \t]*(?P=fence)[~`]*[ \t]*$|\Z)",
    re.DOTALL | re.MULTILINE,
)

PYTHON_LANGS = {"python", "py", "python3", "py3"}

def find_code_blocks(text: str) -> list[tuple[str, str]]:
    """Devuelve ``(lenguaje, código)`` de cada bloque delimitado por ``` o ~~~."""
    text = text.replace("\r\n", "\n").replace("\n", "\n")
    blocks = []
    for match in _FENCE_RE.finditer(text):
        body = match.group("body")
        if body.endswith("\n"):
            body = body[:-1]
        # Un bloque dentro de una lista viene sangrado entero: se quita la sangría común.
        blocks.append((match.group("lang").lower(), textwrap.dedent(body)))
    return blocks

def extract_code(text: str) -> str | None:
    """Primer bloque ```python``; si no hay, el único bloque; si no, el primero sin lenguaje.

    Devuelve ``None`` si la respuesta no contiene ningún bloque de código utilizable.
    """
    if not text:
        return None
    blocks = [(lang, body) for lang, body in find_code_blocks(text) if body.strip()]
    code: str | None = None
    for lang, body in blocks:
        if lang in PYTHON_LANGS:
            code = body
            break
    else:
        if len(blocks) == 1:
            code = blocks[0][1]
        else:
            code = next((body for lang, body in blocks if not lang), None)
    if code is None or not code.strip():
        return None
    return code.strip("\n") + "\n"

```

src/modelduel/runner.py · 373 líneas

```
"""Construye el prompt y ejecuta los tests de una tarea sobre el código de un modelo.

El código generado se ejecuta SIEMPRE en un directorio temporal, en un subprocesso con límite
de tiempo y sin las variables de entorno que parecen secretos. Al terminar (o al agotar el
tiempo) se mata el árbol de procesos completo: un *Job Object* en Windows y un grupo de procesos
en POSIX. La salida va a un archivo y solo se leen su principio y su final, así que una solución
que imprime sin parar no puede agotar la memoria.
"""

from __future__ import annotations

import contextlib
import importlib.util
import os
import re
import shutil
import signal
import subprocess
import sys
import tempfile
import time
import xml.etree.ElementTree as ET
from dataclasses import asdict, dataclass
from pathlib import Path

from modelduel.tasks import TEST_FILE, Task

DEFAULT_TIMEOUT = 20.0
MAX_OUTPUT_CHARS = 12_000
# Bytes que se leen como máximo del archivo de salida (mitad del principio, mitad del final).
MAX_READ_BYTES = 256 * 1024

PROMPT_INSTRUCTIONS = """\
---
Instrucciones de formato:
- Responde con UN ÚNICO bloque de código Python delimitado por ```python y ```.
- El bloque debe contener la implementación completa y autocontenida (solo biblioteca estándar).
- No incluyas tests ni código que se ejecute al importar el módulo.
- Respeta exactamente el nombre y la firma de la función pedida."""

_SECRET_HINTS = ("KEY", "TOKEN", "SECRET", "PASSWORD", "CREDENTIAL")

class RunnerError(Exception):
    """Error de configuración del runner (no del código del modelo)."""

@dataclass
class TestRun:
    """Resultado de ejecutar los tests de una tarea sobre una solución."""

    status: str # ok | no_code | import_error | timeout | error
    passed: int = 0
    failed: int = 0
    errors: int = 0
    skipped: int = 0
    total: int = 0
    duration_s: float = 0.0
    output: str = ""
    message: str = ""

    @property
    def solved(self) -> bool:
        return self.status == "ok" and self.total > 0 and self.passed == self.total

    def to_dict(self) -> dict:
        data = asdict(self)
        data["solved"] = self.solved
        return data

def build_prompt(task: Task) -> str:
    return f"{task.statement}\n\n{PROMPT_INSTRUCTIONS}\n"

def ensure_pytest_available() -> None:
```

```

if importlib.util.find_spec("pytest") is None:
    raise RunnerError(
        "modelduel necesita pytest para ejecutar los tests de las tareas. "
        "Instálalo en este entorno con: pip install pytest"
    )

def safe_env() -> dict[str, str]:
    """Entorno para el subprocesso sin claves ni configuración de pytest ajena."""
    env = {
        name: value
        for name, value in os.environ.items()
        if not any(hint in name.upper() for hint in _SECRET_HINTS)
    }
    for name in ("PYTEST_ADDOPTS", "PYTEST_PLUGINS", "PYTHONPATH", "PYTHONSTARTUP"):
        env.pop(name, None)
    env["PYTHONDONTWRITEBYTECODE"] = "1"
    env["PYTHONIOENCODING"] = "utf-8"
    env["PYTEST_DISABLE_PLUGIN_AUTOLOAD"] = "1"
    return env

def run_tests(code: str | None, task: Task, timeout: float = DEFAULT_TIMEOUT) -> TestRun:
    """Ejecuta ``test_task.py`` contra ``code`` guardado como ``solution.py``."""
    expected = task.expected_tests
    if code is None:
        return TestRun(
            status="no_code",
            total=expected,
            message="La respuesta no contiene ningún bloque de código.",
        )

    root = Path(tempfile.mkdtemp(prefix="modelduel-"))
    try:
        workdir = root / "work"
        tmpdir = root / "tmp"
        workdir.mkdir()
        tmpdir.mkdir()
        (workdir / "solution.py").write_text(code, encoding="utf-8", errors="replace")
        shutil.copyfile(task.test_path, workdir / TEST_FILE)
        conftest = task.path / "conftest.py"
        if conftest.is_file():
            shutil.copyfile(conftest, workdir / "conftest.py")
        # Un pytest.ini vacío aísla la ejecución de cualquier configuración del usuario.
        (workdir / "pytest.ini").write_text("[pytest]\n", encoding="utf-8")
        junit = root / "junit.xml"
        output_path = root / "output.log"
        cmd = [
            sys.executable,
            "-m",
            "pytest",
            "-q",
            "--tb=short",
            "-p",
            "no:cacheprovider",
            f"--junitxml={junit}",
            TEST_FILE,
        ]
        env = safe_env()
        # Los temporales de la solución y de pytest se quedan dentro de la ejecución y se borran.
        for name in ("TMP", "TEMP", "TMPDIR"):
            env[name] = str(tmpdir)
        returncode, duration = _execute(cmd, workdir, env, output_path, timeout)
        output = _truncate(_clean_output(_read_capped(output_path), root))
        counts = parse_junit(junit) if junit.is_file() else None
    finally:
        _remove_tree(root)

    if returncode is None:
        return TestRun(
            status="timeout",
            total=expected,
            duration_s=duration,
            output=output,
            message=f"Los tests superaron el límite de {timeout:g} s y se interrumpieron.",
        )
    if returncode == 5 and (counts is None or counts["total"] == 0):

```

```

        return TestRun(
            status="error",
            total=expected,
            duration_s=duration,
            output=output,
            message="pytest no encontró ningún test en la tarea: revisa su test_task.py.",
        )
    if counts is None:
        return TestRun(
            status="error",
            total=expected,
            duration_s=duration,
            output=output,
            message=f"pytest terminó con código {returncode} sin generar resultados.",
        )
    if counts["collection_error"] or (returncode in (2, 3, 4) and counts["total"] == 0):
        return TestRun(
            status="import_error",
            total=expected,
            errors=max(counts["errors"], 1),
            duration_s=duration,
            output=output,
            message="No se pudo importar la solución o recoger los tests.",
        )
    total = counts["total"] or expected
    return TestRun(
        status="ok",
        passed=counts["passed"],
        failed=counts["failed"],
        errors=counts["errors"],
        skipped=counts["skipped"],
        total=total,
        duration_s=duration,
        output=output,
    )

def parse_junit(path: Path) -> dict | None:
    """Cuenta resultados leyendo el XML JUnit de pytest."""
    try:
        root = ET.parse(path).getroot()
    except ET.ParseError:
        return None
    counts = {"passed": 0, "failed": 0, "errors": 0, "skipped": 0, "total": 0}
    collection_error = False
    for case in root.iter("testcase"):
        tags = {child.tag for child in case}
        if "error" in tags:
            counts["errors"] += 1
            # Los errores de recogida (p. ej. ImportError) aparecen sin nombre de clase.
            if not case.get("classname"):
                collection_error = True
        elif "failure" in tags:
            counts["failed"] += 1
        elif "skipped" in tags:
            counts["skipped"] += 1
        else:
            counts["passed"] += 1
    if collection_error:
        counts["total"] = 0
        counts["passed"] = counts["failed"] = counts["skipped"] = 0
    else:
        counts["total"] = sum(counts[k] for k in ("passed", "failed", "errors", "skipped"))
    counts["collection_error"] = collection_error
    return counts

# ----- subprocesso

def _execute(
    cmd: list[str], workdir: Path, env: dict[str, str], output_path: Path, timeout: float
) -> tuple[int | None, float]:
    """Ejecuta ``cmd`` y devuelve ``(código de salida, duración)``; ``None`` si agota el tiempo.

    stdout y stderr van juntos a ``output_path``: un archivo no obliga a esperar a que los
    procesos nietos suelten una tubería. Al volver no queda vivo ningún proceso del árbol

```

```

(salvo los que se desliguen a propósito de él; ver «Problemas conocidos» en MEMORY.md).
"""
extra: dict = {}
if os.name != "nt":
    extra["start_new_session"] = True # grupo de procesos propio para matarlo entero
start = time.perf_counter()
with open(output_path, "wb") as out:
    proc = subprocess.Popen(
        cmd,
        cwd=workdir,
        env=env,
        stdin=subprocess.DEVNULL,
        stdout=out,
        stderr=subprocess.STDOUT,
        **extra,
    )
    tree = _ProcessTree(proc)
    try:
        returncode: int | None = proc.wait(timeout=timeout)
    except subprocess.TimeoutExpired:
        returncode = None
    finally:
        # También tras un final normal: la solución puede haber dejado hijos en marcha.
        tree.kill()
        proc.wait()
return returncode, round(time.perf_counter() - start, 3)

class _ProcessTree:
    """Mata un proceso y todos sus descendientes solo con la biblioteca estándar."""

    def __init__(self, proc: subprocess.Popen) -> None:
        self.proc = proc
        self.job = _win_job_for(proc) if os.name == "nt" else None

    def kill(self) -> None:
        if os.name != "nt":
            with contextlib.suppress(ProcessLookupError, PermissionError):
                os.killpg(self.proc.pid, signal.SIGKILL)
            return
        if self.job is not None: # pragma: no cover - solo Windows
            _win_terminate_job(self.job)
            self.job = None
        elif self.proc.poll() is None: # pragma: no cover - solo Windows sin Job Object
            # taskkill recorre el árbol mientras el proceso padre siga vivo.
            subprocess.run(
                ["taskkill", "/F", "/T", "/PID", str(self.proc.pid)],
                stdout=subprocess.DEVNULL,
                stderr=subprocess.DEVNULL,
                check=False,
            )

    def _kernel32(): # pragma: no cover - solo Windows
        import ctypes
        from ctypes import wintypes

        kernel32 = ctypes.WinDLL("kernel32", use_last_error=True)
        kernel32.CreateJobObjectW.argtypes = (wintypes.LPVOID, wintypes.LPCWSTR)
        kernel32.CreateJobObjectW.restype = wintypes.HANDLE
        kernel32.AssignProcessToJobObject.argtypes = (wintypes.HANDLE, wintypes.HANDLE)
        kernel32.AssignProcessToJobObject.restype = wintypes.BOOL
        kernel32.TerminateJobObject.argtypes = (wintypes.HANDLE, wintypes.UINT)
        kernel32.TerminateJobObject.restype = wintypes.BOOL
        kernel32.CloseHandle.argtypes = (wintypes.HANDLE,)
        kernel32.CloseHandle.restype = wintypes.BOOL
        return kernel32

    def _win_job_for(proc: subprocess.Popen) -> int | None: # pragma: no cover - solo Windows
        """Mete ``proc`` en un Job Object nuevo (sus hijos entran solos). ``None`` si no se puede."""
        try:
            kernel32 = _kernel32()
            job = kernel32.CreateJobObjectW(None, None)
            if not job:
                return None
            if not kernel32.AssignProcessToJobObject(job, int(proc._handle)): # type: ignore[attr-defined]

```

```

        kernel32.CloseHandle(job)
        return None
    return job
except (OSError, AttributeError, ValueError):
    return None

def _win_terminate_job(job: int) -> None: # pragma: no cover - solo Windows
    kernel32 = _kernel32()
    kernel32.TerminateJobObject(job, 1)
    kernel32.CloseHandle(job)

def _remove_tree(path: Path) -> None:
    """Borra el temporal; en Windows reintenta porque un proceso recién muerto tarda en soltarlo."""
    for _ in range(10):
        shutil.rmtree(path, ignore_errors=True)
        if not path.exists():
            return
        time.sleep(0.2)

# ----- salida

def _read_capped(path: Path, limit: int = MAX_READ_BYTES) -> str:
    """Lee como mucho ``limit`` bytes del archivo: su principio y su final."""
    try:
        size = path.stat().st_size
        with open(path, "rb") as fh:
            if size <= limit:
                return fh.read().decode("utf-8", errors="replace")
            half = limit // 2
            head = fh.read(half)
            fh.seek(size - half)
            tail = fh.read(half)
    except OSError:
        return ""
    return (
        head.decode("utf-8", errors="replace")
        + f"\n... [salida recortada: {size - 2 * half} bytes omitidos] ...\n"
        + tail.decode("utf-8", errors="replace")
    )

def _clean_output(output: str, root: Path) -> str:
    """Normaliza los finales de línea y quita la ruta del temporal para no mostrar rutas locales."""
    output = output.replace("\r\n", "\n").replace("\r", "\n")
    variants: set[str] = set()
    for base in (root / "work", root):
        for form in (base, base.resolve()):
            variants |= {str(form), form.as_posix()}
    # Primero las más largas, para que ``<root>/work`` no quede como ``<tmp>/work``.
    for variant in sorted(variants, key=len, reverse=True):
        output = output.replace(variant, "<tmp>")
    return re.sub(r"\n{3,}", "\n\n", output).strip()

def _truncate(text: str, limit: int = MAX_OUTPUT_CHARS) -> str:
    """Conserva el principio y el final: pytest imprime el resumen al final."""
    if len(text) <= limit:
        return text
    half = limit // 2
    omitted = len(text) - 2 * half
    return f"{text[:half]}\n... [salida recortada: {omitted} caracteres omitidos] ...\n{text[-half:]}"

```

src/modelduel/pricing.py · 95 líneas

```
"""Coste estimado a partir de tarifas por millón de tokens.

coste = entrada / 1e6 × tarifa_entrada + salida / 1e6 × tarifa_salida

La tabla integrada solo contiene precios FICTICIOS para los contendientes ``replay`` de la demo.
Los precios reales cambian a menudo: añádelos tú con ``--prices precios.json``:

{
  "openai:mi-modelo": {"input": 0.15, "output": 0.60, "currency": "USD"},
  "gemini:otro-modelo": {"input": 0.10, "output": 0.40, "currency": "EUR"}
}

Si un modelo no tiene precio, el coste es «sin datos»: nunca se inventa.
"""

from __future__ import annotations

import json
from dataclasses import asdict, dataclass
from pathlib import Path

class PricingError(Exception):
    """Archivo de precios no válido."""

@dataclass(frozen=True)
class Price:
    input: float # por millón de tokens de entrada
    output: float # por millón de tokens de salida
    currency: str = "USD"
    fictitious: bool = False
    source: str = "builtin"

    def to_dict(self) -> dict:
        return asdict(self)

BUILTIN_PRICES: dict[str, Price] = {
    # Precios FICTICIOS: solo sirven para que la demo con respuestas grabadas muestre costes.
    "replay:alfa": Price(input=0.40, output=1.60, currency="USD", fictitious=True),
    "replay:beta": Price(input=2.50, output=10.00, currency="USD", fictitious=True),
}

def load_prices(path: Path | None) -> dict[str, Price]:
    """Tabla integrada ampliada (o sobrescrita) con el JSON del usuario."""
    table = dict(BUILTIN_PRICES)
    if path is None:
        return table
    try:
        data = json.loads(Path(path).read_text(encoding="utf-8-sig"))
    except OSError as exc:
        raise PricingError(f"No se pudo leer {path}: {exc}") from exc
    except json.JSONDecodeError as exc:
        raise PricingError(f"{path} no es JSON válido: {exc}") from exc
    if not isinstance(data, dict):
        raise PricingError(f'{path} debe ser un objeto JSON {"proveedor:modelo": {{...}}}.')
    for key, entry in data.items():
        if key.startswith("_"): # permite comentarios como "_nota"
            continue
        if not isinstance(entry, dict):
            raise PricingError(f"El precio de «{key}» debe ser un objeto con input y output.")
        try:
            price_in = float(entry["input"])
            price_out = float(entry["output"])
        except (KeyError, TypeError, ValueError) as exc:
            raise PricingError(f"El precio de «{key}» necesita input y output numéricos.") from exc
        if price_in < 0 or price_out < 0:
            raise PricingError(f"El precio de «{key}» no puede ser negativo.")
        table[key] = Price(
            input=price_in,
            output=price_out,
            currency=str(entry.get("currency", "USD")).upper(),
            fictitious=bool(entry.get("fictitious", False)),
            source="file",
        )
```

```

    )
    return table

def find_price(spec: str, table: dict[str, Price]) -> Price | None:
    """Busca por ``proveedor:modelo`` y, si no, solo por ``modelo``."""
    if spec in table:
        return table[spec]
    _, _, model = spec.partition(":")
    return table.get(model) if model else None

def compute_cost(
    price: Price | None, input_tokens: int | None, output_tokens: int | None
) -> float | None:
    if price is None or input_tokens is None or output_tokens is None:
        return None
    return input_tokens / 1e6 * price.input + output_tokens / 1e6 * price.output

```

src/modelduel/results.py · 76 líneas

```
"""Estructura de ``results.json`` y resumen tipo marcador."""

from __future__ import annotations

import json
from pathlib import Path

SCHEMA_VERSION = 1
SIDES = ("a", "b")

class ResultsError(Exception):
    """``results.json`` ilegible o con formato inesperado."""

def summarize(results: dict) -> dict[str, dict]:
    """Totales por contendiente a partir de los intentos registrados."""
    summary: dict[str, dict] = {}
    tasks = results.get("tasks", [])
    for side in SIDES:
        attempts = [a for task in tasks for a in task["results"].get(side, [])]
        solved_tasks = sum(
            1
            for task in tasks
            if task["results"].get(side) and all(a["solved"] for a in task["results"][side])
        )
        costs = [a.get("cost") for a in attempts]
        inputs = [a.get("input_tokens") for a in attempts]
        outputs = [a.get("output_tokens") for a in attempts]
        price = (results.get("contenders", {}).get(side) or {}).get("price")
        summary[side] = {
            "tests_passed": sum(a["passed"] for a in attempts),
            "tests_total": sum(a["total"] for a in attempts),
            "tasks_solved": solved_tasks,
            "tasks_total": len(tasks),
            "attempts_solved": sum(1 for a in attempts if a["solved"]),
            "attempts_total": len(attempts),
            "latency_s": round(sum(a.get("latency_s") or 0.0 for a in attempts), 3),
            "test_time_s": round(sum(a.get("duration_s") or 0.0 for a in attempts), 3),
            "input_tokens": _sum_known(inputs),
            "output_tokens": _sum_known(outputs),
            "tokens_complete": all(v is not None for v in inputs + outputs),
            "cost": sum(costs) if attempts and all(c is not None for c in costs) else None,
            "currency": price["currency"] if price else None,
            "fictitious_price": bool(price and price.get("fictitious")),
        }
    return summary

def _sum_known(values: list[int | None]) -> int | None:
    known = [v for v in values if v is not None]
    return sum(known) if known else None

def save_results(results: dict, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(results, ensure_ascii=False, indent=2) + "\n", encoding="utf-8")

def load_results(path: Path) -> dict:
    try:
        data = json.loads(Path(path).read_text(encoding="utf-8-sig"))
    except OSError as exc:
        raise ResultsError(f"No se pudo leer {path}: {exc}") from exc
    except json.JSONDecodeError as exc:
        raise ResultsError(f"{path} no es JSON válido: {exc}") from exc
    if not isinstance(data, dict) or "tasks" not in data or "contenders" not in data:
        raise ResultsError(f"{path} no parece un results.json de modelduel.")
    try:
        data["summary"] = summarize(data)
    except (KeyError, TypeError, ValueError, AttributeError) as exc:
        raise ResultsError(
            f"{path} no parece un results.json de modelduel ({type(exc).__name__}: {exc})."
        ) from exc
    return data
```

src/modelduel/providers/base.py · 149 líneas

```
"""Interfaz común de los proveedores y utilidades HTTP con ``urllib``."""

from __future__ import annotations

import http.client
import json
import math
import os
import time
import urllib.error
import urllib.request
from dataclasses import dataclass
from typing import Protocol

DEFAULT_HTTP_TIMEOUT = 180.0
USER_AGENT = "modelduel (+https://github.com/BertMarti/modelduel)"

class ProviderError(Exception):
    """Error de un proveedor (configuración, red o respuesta inesperada)."""

@dataclass
class Response:
    text: str
    input_tokens: int | None
    output_tokens: int | None
    latency_s: float

class Provider(Protocol):
    spec: str

    def complete(self, prompt: str, *, task_id: str | None = None) -> Response:
        """Envía ``prompt`` al modelo. ``task_id`` solo lo usan los proveedores grabados."""
        ...

    def require_env(name: str, hint: str) -> str:
        value = os.environ.get(name, "").strip()
        if not value:
            raise ProviderError(f"Falta la variable de entorno {name}. {hint}")
        return value

    def http_timeout() -> float:
        """Límite de las peticiones HTTP (``MODEL_DUEL_HTTP_TIMEOUT``), validado al usarlo."""
        raw = os.environ.get("MODEL_DUEL_HTTP_TIMEOUT", "").strip()
        if not raw:
            return DEFAULT_HTTP_TIMEOUT
        try:
            value = float(raw.replace(",", "."))
        except ValueError:
            value = math.nan
        if not math.isfinite(value) or value <= 0:
            raise ProviderError(
                f"MODEL_DUEL_HTTP_TIMEOUT debe ser un número de segundos mayor que 0 (vale «{raw}»).")
        return value

_HTTP_HINTS = {
    400: "petición rechazada: revisa el nombre del modelo",
    401: "revisa la clave de la API",
    403: "revisa la clave y los permisos de la API",
    404: "¿existe el modelo y es correcta la URL base?",
    429: "límite de peticiones o cuota agotada: espera o revisa tu plan",
}

def post_json(
    url: str, payload: dict, headers: dict[str, str], secrets: tuple[str, ...] = ()
) -> tuple[dict, float]:
    """POST JSON y devuelve ``(respuesta, latencia_s)``. Nunca incluye secretos en los errores.

    Cualquier fallo de red o de protocolo se convierte en ``ProviderError`` para que el duelo
```

```

lo registre como «error del proveedor» y siga con el resto de intentos.
"""
timeout = http_timeout()
body = json.dumps(payload).encode("utf-8")
request = urllib.request.Request(
    url,
    data=body,
    method="POST",
    headers={"Content-Type": "application/json", "User-Agent": USER_AGENT, **headers},
)
start = time.perf_counter()
try:
    with urllib.request.urlopen(request, timeout=timeout) as resp:
        raw = resp.read()
except urllib.error.HTTPError as exc:
    detail = error_detail(exc)
    hint = _HTTP_HINTS.get(exc.code) or ("error del servidor" if exc.code >= 500 else "")
    hint = f" ({hint})" if hint else ""
    raise ProviderError(redact(f"HTTP {exc.code} en {url}{hint}: {detail}", secrets)) from None
except urllib.error.URLError as exc:
    if isinstance(exc.reason, TimeoutError):
        raise ProviderError(f"La petición a {url} superó {timeout:g} s.") from None
    raise ProviderError(
        redact(f"No se pudo conectar con {url}: {exc.reason}", secrets)
    ) from None
except TimeoutError:
    raise ProviderError(f"La petición a {url} superó {timeout:g} s.") from None
except (OSError, http.client.HTTPException) as exc:
    # p. ej. RemoteDisconnected, ConnectionResetError o IncompleteRead al leer la respuesta.
    name = type(exc).__name__
    raise ProviderError(redact(f"Se cortó la conexión con {url} ({name}).", secrets)) from None
latency = time.perf_counter() - start
try:
    return json.loads(raw.decode("utf-8")), latency
except (UnicodeDecodeError, json.JSONDecodeError) as exc:
    raise ProviderError(f"Respuesta no JSON de {url}: {exc}") from None

def error_detail(exc: urllib.error.HTTPError, limit: int = 300) -> str:
    """Mensaje legible de un error HTTP: el ``error.message`` del JSON si lo hay."""
    try:
        text = exc.read().decode("utf-8", errors="replace")
    except (OSError, http.client.HTTPException):
        return str(exc.reason or "sin detalle")
    try:
        data = json.loads(text)
    except json.JSONDecodeError:
        data = None
    if isinstance(data, list) and data: # Gemini a veces envuelve el error en una lista
        data = data[0]
    error = data.get("error") if isinstance(data, dict) else None
    if isinstance(error, str) and error.strip():
        return error.strip()[:limit]
    if isinstance(error, dict) and error.get("message"):
        status = error.get("status") or error.get("type") or error.get("code")
        message = str(error["message"]).strip()
        return (f"{status}: {message}" if status else message)[:limit]
    return " ".join(text.split()[:limit] or str(exc.reason or "sin detalle"))

def redact(message: str, secrets: tuple[str, ...]) -> str:
    for secret in secrets:
        if secret:
            message = message.replace(secret, "****")
    return message

def as_int(value: object) -> int | None:
    try:
        return int(value) # type: ignore[arg-type]
    except (TypeError, ValueError):
        return None

```

src/modelduel/providers/__init__.py · 35 líneas

```
"""Proveedores de modelos. Formato de especificación: ``proveedor:modelo``."""

from __future__ import annotations

from pathlib import Path

from modelduel.providers.base import Provider, ProviderError, Response
from modelduel.providers.gemini import GeminiProvider
from modelduel.providers.openai_compat import OpenAIProvider
from modelduel.providers.replay import ReplayProvider

PROVIDERS = ("replay", "gemini", "openai")

__all__ = ["Provider", "ProviderError", "Response", "get_provider", "parse_spec", "PROVIDERS"]

def parse_spec(spec: str) -> tuple[str, str]:
    kind, sep, model = spec.partition(":")
    kind = kind.strip().lower()
    model = model.strip()
    if not sep or not model:
        raise ProviderError(f"Especificación «{spec}» no válida: usa proveedor:modelo.")
    if kind not in PROVIDERS:
        raise ProviderError(f"Proveedor «{kind}» desconocido. Disponibles: {' ', ' '.join(PROVIDERS)}.")
    return kind, model

def get_provider(spec: str, replay_dirs: list[Path] | None = None) -> Provider:
    kind, model = parse_spec(spec)
    if kind == "replay":
        return ReplayProvider(model, replay_dirs)
    if kind == "gemini":
        return GeminiProvider(model)
    return OpenAIProvider(model)
```

src/modelduel/providers/replay.py · 77 líneas

```
"""Proveedor ``replay``: respuestas grabadas en ``<replays>/<nombre>/<tarea>.md``.
```

Cada archivo puede empezar con un front-matter sencillo::

```
---
input_tokens: 312
output_tokens: 188
latency_s: 2.41
---
Texto de la respuesta del modelo...
"""
```

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
from modelduel.providers.base import ProviderError, Response, as_int
```

```
def parse_front_matter(text: str) -> tuple[dict[str, str], str]:
    text = text.replace("\r\n", "\n").rstrip("")
    if not text.startswith("---\n"):
        return {}, text
    end = text.find("\n---", 4)
    if end == -1:
        return {}, text
    meta: dict[str, str] = {}
    for line in text[4:end].splitlines():
        key, sep, value = line.partition(":")
        if sep and key.strip() and not key.strip().startswith("#"):
            meta[key.strip()] = value.strip()
    body = text[end + 4 :]
    if body.startswith("\n"):
        body = body[1:]
    return meta, body
```

```
class ReplayProvider:
```

```
    def __init__(self, name: str, search_dirs: list[Path] | None = None) -> None:
        if not name:
            raise ProviderError("replay necesita un nombre: replay:<nombre>.")
        self.name = name
        self.spec = f"replay:{name}"
        self.directory = self._resolve(name, search_dirs or [Path("examples/replays")])
```

```
    @staticmethod
```

```
    def _resolve(name: str, search_dirs: list[Path]) -> Path:
        direct = Path(name)
        if direct.is_dir():
            return direct
        for base in search_dirs:
            candidate = Path(base) / name
            if candidate.is_dir():
                return candidate
        places = ", ".join(str(Path(b) / name) for b in search_dirs)
        raise ProviderError(
            f"No encuentro las respuestas grabadas de «{name}» (busqué en {places})."
        )
```

```
    def complete(self, prompt: str, *, task_id: str | None = None) -> Response:
        if not task_id:
            raise ProviderError("replay necesita saber la tarea para leer su respuesta grabada.")
        path = self.directory / f"{task_id}.md"
        if not path.is_file():
            raise ProviderError(f"No hay respuesta grabada para la tarea «{task_id}» en {path}.")
        meta, body = parse_front_matter(path.read_text(encoding="utf-8"))
        try:
            latency = float(meta.get("latency_s", "0").replace(",", "."))
        except ValueError:
            latency = 0.0
        return Response(
            text=body,
            input_tokens=as_int(meta.get("input_tokens")),
            output_tokens=as_int(meta.get("output_tokens")),
            latency_s=latency,
        )
```

src/modelduel/providers/gemini.py · 68 líneas

```
"""Proveedor ``gemini:<modelo>``: API REST ``generateContent`` de Google Generative Language."""

from __future__ import annotations

import os
import urllib.parse

from modelduel.providers.base import ProviderError, Response, as_int, post_json, require_env

DEFAULT_BASE_URL = "https://generativelanguage.googleapis.com/v1beta"

class GeminiProvider:
    def __init__(self, model: str) -> None:
        if not model:
            raise ProviderError("gemini necesita un modelo: gemini:<modelo>.")
        self.model = model
        self.spec = f"gemini:{model}"
        self.base_url = os.environ.get("GEMINI_BASE_URL", DEFAULT_BASE_URL).rstrip("/")
        self.api_key = require_env(
            "GEMINI_API_KEY", "Crea una clave en Google AI Studio y expórtala en tu terminal."
        )

    def complete(self, prompt: str, *, task_id: str | None = None) -> Response:
        model = urllib.parse.quote(self.model, safe="-.")
        url = f"{self.base_url}/models/{model}:generateContent"
        payload = {"contents": [{"role": "user", "parts": [{"text": prompt}]}]}
        # La clave va en una cabecera, no en la URL, para que no aparezca en errores ni logs.
        data, latency = post_json(
            url, payload, {"x-goog-api-key": self.api_key}, secrets=(self.api_key,)
        )
        return parse_gemini_response(data, latency)

def parse_gemini_response(data: object, latency: float) -> Response:
    if not isinstance(data, dict):
        raise ProviderError("Gemini devolvió una respuesta con un formato inesperado.")
    candidates = data.get("candidates") or []
    if not candidates:
        feedback = data.get("promptFeedback")
        reason = feedback.get("blockReason") if isinstance(feedback, dict) else None
        raise ProviderError(f"Gemini no devolvió respuesta ({reason or 'sin candidatos'}).")
    first = candidates[0]
    if not isinstance(first, dict):
        raise ProviderError("Gemini devolvió un candidato con un formato inesperado.")
    content = first.get("content")
    parts = content.get("parts") if isinstance(content, dict) else None
    text = "".join(
        str(p.get("text") or "")
        for p in parts or []
        if isinstance(p, dict) and not p.get("thought")
    )
    if not text.strip():
        # SAFETY, RECITATION, MAX_TOKENS (todo el presupuesto en razonamiento)...
        reason = first.get("finishReason") or "sin texto"
        raise ProviderError(f"Gemini no devolvió texto (finishReason: {reason}).")
    usage = data.get("usageMetadata")
    usage = usage if isinstance(usage, dict) else {}
    output = as_int(usage.get("candidatesTokenCount"))
    thoughts = as_int(usage.get("thoughtsTokenCount")) or 0
    return Response(
        text=text,
        input_tokens=as_int(usage.get("promptTokenCount")),
        # Los tokens de razonamiento se facturan como salida.
        output_tokens=None if output is None else output + thoughts,
        latency_s=latency,
    )
```

src/modelduel/providers/openai_compat.py · 77 líneas

```
"""Proveedor ``openai:<modelo>``: cualquier API compatible con Chat Completions de OpenAI.

Sirve para OpenAI, OpenRouter, Ollama y similares cambiando ``OPENAI_BASE_URL``.
"""

from __future__ import annotations

import os
import urllib.parse

from modelduel.providers.base import ProviderError, Response, as_int, post_json

DEFAULT_BASE_URL = "https://api.openai.com/v1"
_LOCAL_HOSTS = {"localhost", "127.0.0.1", "::1", "0.0.0.0"}

class OpenAIProvider:
    def __init__(self, model: str) -> None:
        if not model:
            raise ProviderError("openai necesita un modelo: openai:<modelo>.")
        self.model = model
        self.spec = f"openai:{model}"
        self.base_url = (os.environ.get("OPENAI_BASE_URL") or DEFAULT_BASE_URL).rstrip("/")
        self.api_key = os.environ.get("OPENAI_API_KEY", "").strip()
        host = urllib.parse.urlparse(self.base_url).hostname or ""
        if not self.api_key and host not in _LOCAL_HOSTS:
            raise ProviderError(
                "Falta la variable de entorno OPENAI_API_KEY "
                f"(necesaria para {self.base_url}; los servidores locales como Ollama no la piden)."
            )

    def complete(self, prompt: str, *, task_id: str | None = None) -> Response:
        headers = {}
        if self.api_key:
            headers["Authorization"] = f"Bearer {self.api_key}"
        payload = {"model": self.model, "messages": [{"role": "user", "content": prompt}]}
        data, latency = post_json(
            f"{self.base_url}/chat/completions", payload, headers, secrets=(self.api_key,)
        )
        return parse_openai_response(data, latency)

def parse_openai_response(data: object, latency: float) -> Response:
    if not isinstance(data, dict):
        raise ProviderError("La API devolvió una respuesta con un formato inesperado.")
    choices = data.get("choices") or []
    if not choices:
        error = data.get("error")
        if isinstance(error, dict):
            detail = error.get("message") or "sin opciones"
        else:
            detail = error if isinstance(error, str) and error.strip() else "sin opciones"
        raise ProviderError(f"La API no devolvió respuesta ({detail}).")
    choice = choices[0]
    if not isinstance(choice, dict):
        raise ProviderError("La API devolvió una opción con un formato inesperado.")
    message = choice.get("message")
    message = message if isinstance(message, dict) else {}
    content = message.get("content") or ""
    if isinstance(content, list): # algunos servidores devuelven partes
        content = "".join(str(p.get("text") or "") for p in content if isinstance(p, dict))
    if not isinstance(content, str):
        content = ""
    if not content.strip():
        if message.get("refusal"):
            raise ProviderError(f"El modelo rechazó la petición: {message['refusal']}")
        reason = choice.get("finish_reason") or "sin texto"
        raise ProviderError(f"La API no devolvió texto (finish_reason: {reason}).")
    usage = data.get("usage")
    usage = usage if isinstance(usage, dict) else {}
    return Response(
        text=content,
        input_tokens=as_int(usage.get("prompt_tokens")),
        output_tokens=as_int(usage.get("completion_tokens")),
        latency_s=latency,
    )
```


src/modelduel/report/html.py · 468 líneas

```
"""Informe HTML autocontenido: ``string.Template``, CSS en línea, SVG en línea y sin JavaScript."""

from __future__ import annotations

from datetime import datetime
from html import escape
from importlib import resources
from pathlib import Path
from string import Template

from modelduel import __version__
from modelduel.results import SIDES, summarize

STATUS_LABELS = {
    "ok": "tests ejecutados",
    "no_code": "sin bloque de código",
    "import_error": "no se pudo importar",
    "timeout": "tiempo agotado",
    "error": "error al ejecutar",
    "provider_error": "error del proveedor",
}

MONTHS = ["ene", "feb", "mar", "abr", "may", "jun", "jul", "ago", "sep", "oct", "nov", "dic"]

# ----- formato

def e(value: object) -> str:
    return escape(" if value is None else str(value), quote=True)

def fmt_int(value: int | None) -> str:
    if value is None:
        return "sin datos"
    return f"{value:,}".replace(",", ".")

def fmt_seconds(value: float | None) -> str:
    if value is None:
        return "sin datos"
    value = round(value, 1) # sin esto, 119,97 s salía como «1 min 60,0 s»
    if value >= 60:
        minutes, seconds = divmod(value, 60)
        return f"{int(minutes)} min {seconds:04.1f} s".replace(".", ",")
    return f"{value:.1f} s".replace(".", ",")

def fmt_cost(value: float | None, currency: str | None) -> str:
    if value is None:
        return "sin datos"
    currency = currency or ""
    if value == 0:
        text = "0"
    elif value < 0.0001:
        text = "<0,0001"
    elif value < 1:
        text = f"{value:.4f}".replace(".", ",")
    else:
        text = f"{value:,.2f}".replace(",", "X").replace(".", ",").replace("X", ".")
    return f"{text} {currency}".strip()

def fmt_tokens(inp: int | None, out: int | None) -> str:
    if inp is None and out is None:
        return "sin datos"
    return f"{fmt_int(inp)} / {fmt_int(out)}"

def fmt_date(iso: str) -> str:
    try:
        dt = datetime.fromisoformat(iso)
    except (TypeError, ValueError):
        return str(iso) # texto plano: lo escapa quien lo inserta
    return f"{dt.day} {MONTHS[dt.month - 1]} {dt.year} · {dt:%H:%M} UTC"
def plural(n: int, one: str, many: str) -> str:
```

```

    return f"{n} {one if n == 1 else many}"

# ----- SVG

def svg_versus_bar(value_a: float | None, value_b: float | None, label: str) -> str:
    """Barras enfrentadas desde el centro: A crece a la izquierda, B a la derecha."""
    a = value_a or 0.0
    b = value_b or 0.0
    top = max(a, b) or 1.0
    half = 200
    gap = 2
    len_a = (half - gap) * a / top
    len_b = (half - gap) * b / top
    return (
        f'<svg viewBox="0 0 400 6" preserveAspectRatio="none" role="img" aria-label="{e(label)}">'
        f'<title>{e(label)}</title>'
        f'<rect class="track" x="0" y="2.5" width="400" height="1" fill="#2a2a2f"/>'
        f'<rect class="fill-a" x="{half - gap - len_a:.2f}" y="0" width="{len_a:.2f}" height="6"/>'
        f'<rect class="fill-b" x="{half + gap:.2f}" y="0" width="{len_b:.2f}" height="6"/>'
        "</svg>"
    )

def svg_ratio_bar(ratio: float, side: str, label: str) -> str:
    """Barra fina horizontal (0-100 %) sobre una pista de 1 px."""
    width = max(0.0, min(ratio, 1.0)) * 100
    return (
        f'<svg viewBox="0 0 100 6" preserveAspectRatio="none" role="img" aria-label="{e(label)}">'
        f'<title>{e(label)}</title>'
        f'<rect class="track" x="0" y="2.5" width="100" height="1" fill="#2a2a2f"/>'
        f'<rect class="fill-{side}" x="0" y="0" width="{width:.2f}" height="6"/>'
        "</svg>"
    )

def task_chart(tasks: list[dict]) -> str:
    """Una fila por tarea con dos barras finas (A arriba, B abajo) de tests superados."""
    rows = []
    for task in tasks:
        lines = []
        for side in SIDES:
            attempts = task["results"].get(side, [])
            passed = sum(a["passed"] for a in attempts)
            total = sum(a["total"] for a in attempts)
            ratio = passed / total if total else 0.0
            label = f"{side.upper()}: {passed} de {total} tests superados"
            lines.append(
                f'<div class="bar-line">{svg_ratio_bar(ratio, side, label)}'
                f'<span class="{side}">{passed}/{total}</span></div>'
            )
        diff = f"<span>{e(task.get('difficulty'))}</span>" if task.get("difficulty") else ""
        rows.append(
            f'<div class="bar-row"><div class="bar-title">{e(task["title"])}{diff}</div>'
            + "".join(lines)
            + "</div>"
        )
    return '<div class="bars">' + "".join(rows) + "</div>"

# ----- bloques

def _winner(a: float | None, b: float | None, lower_is_better: bool) -> str | None:
    if a is None or b is None or a == b:
        return None
    if lower_is_better:
        return "a" if a < b else "b"
    return "a" if a > b else "b"

def metric_row(
    label: str,
    shown: tuple[str, str],
    raw: tuple[float | None, float | None],
    lower_is_better: bool = False,

```

```

    subs: tuple[str, str] = ("", ""),
    hint: str = "",
) -> str:
    winner = _winner(row[0], row[1], lower_is_better)
    cells = []
    for idx, side in enumerate(SIDES):
        classes = ["val", side]
        if side == "b":
            classes.append("b-side")
        mark = ""
        if winner == side:
            classes.append("win")
            mark = (
                '<span class="mark" aria-hidden="true">●</span><span class="sr-only">(mejor)</span>'
            )
        elif winner is not None:
            classes.append("lose")
        sub = f'<span class="sub">{e(subs[idx])}</span>' if subs[idx] else ""
        num = f'<span class="num">{e(shown[idx])}</span>'
        value = f"{mark}{num}" if side == "b" else f"{num}{mark}"
        cells.append(f'<div class="{ " ".join(classes) }">{value}{sub}</div>')
    hint_html = f'<div class="hint">{e(hint)}</div>' if hint else ""
    bar = svg_versus_bar(row[0], row[1], f"{label}: A {shown[0]}, B {shown[1]}")
    return (
        f'<div class="metric">{cells[0]}</div>'
        f'<div class="mid"><div class="label">{e(label)}</div>{bar}{hint_html}</div>'
        f'<div class="metric">{cells[1]}</div>'
    )

def build_metrics(summary: dict, runs: int = 1) -> str:
    sa, sb = summary["a"], summary["b"]
    summed = f"suma de {runs} ejecuciones" if runs > 1 else ""
    rows = [
        metric_row(
            "Tareas resueltas",
            (
                f"{sa['tasks_solved']}/{sa['tasks_total']}",
                f"{sb['tasks_solved']}/{sb['tasks_total']}",
            ),
            (sa["tasks_solved"], sb["tasks_solved"]),
            hint="todos los tests en todas las ejecuciones",
        ),
        metric_row(
            "Tests superados",
            (
                f"{sa['tests_passed']}/{sa['tests_total']}",
                f"{sb['tests_passed']}/{sb['tests_total']}",
            ),
            (sa["tests_passed"], sb["tests_passed"]),
            hint=summed,
        ),
        metric_row(
            "Tiempo del modelo",
            (fmt_seconds(sa["latency_s"]), fmt_seconds(sb["latency_s"])),
            (sa["latency_s"], sb["latency_s"]),
            lower_is_better=True,
            subs=(
                f"tests: {fmt_seconds(sa['test_time_s'])}",
                f"tests: {fmt_seconds(sb['test_time_s'])}",
            ),
            hint="menos es mejor" + (f" · {summed}" if summed else ""),
        ),
        metric_row(
            "Tokens",
            (fmt_int(_total_tokens(sa)), fmt_int(_total_tokens(sb))),
            (_total_tokens(sa), _total_tokens(sb)),
            lower_is_better=True,
            subs=(
                f"entrada {fmt_int(sa['input_tokens'])} · salida {fmt_int(sa['output_tokens'])}",
                f"entrada {fmt_int(sb['input_tokens'])} · salida {fmt_int(sb['output_tokens'])}",
            ),
            hint="menos es mejor",
        ),
    ]
    if runs > 1:
        rows.insert(

```

```

1,
metric_row(
    "Intentos resueltos",
    (
        f"{sa['attempts_solved']}/{sa['attempts_total']}",
        f"{sb['attempts_solved']}/{sb['attempts_total']}",
    ),
    (sa["attempts_solved"], sb["attempts_solved"]),
    hint="ejecuciones con todos los tests en verde",
),
)
same_currency = sa["currency"] == sb["currency"]
cost_hint = "menos es mejor"
if sa["fictitious_price"] or sb["fictitious_price"]:
    cost_hint = "precios ficticios de demostración"
elif not same_currency and sa["cost"] is not None and sb["cost"] is not None:
    cost_hint = "monedas distintas: no comparables"
rows.append(
    metric_row(
        "Coste estimado",
        (fmt_cost(sa["cost"], sa["currency"]), fmt_cost(sb["cost"], sb["currency"])),
        (sa["cost"], sb["cost"]) if same_currency else (None, None),
        lower_is_better=True,
        hint=cost_hint,
    )
)
return "\n".join(rows)

def _total_tokens(side: dict) -> int | None:
    if side["input_tokens"] is None and side["output_tokens"] is None:
        return None
    return (side["input_tokens"] or 0) + (side["output_tokens"] or 0)

def _side_task_totals(attempts: list[dict], currency: str | None) -> dict[str, str]:
    passed = sum(a["passed"] for a in attempts)
    total = sum(a["total"] for a in attempts)
    latencies = [a.get("latency_s") for a in attempts]
    inputs = [a.get("input_tokens") for a in attempts]
    outputs = [a.get("output_tokens") for a in attempts]
    costs = [a.get("cost") for a in attempts]
    statuses = {a["status"] for a in attempts} - {"ok"}
    status = ", ".join(STATUS_LABELS.get(s, s) for s in sorted(statuses))
    return {
        "tests": f"{passed}/{total}",
        "status": status,
        "time": fmt_seconds(sum(x for x in latencies if x is not None))
        if any(x is not None for x in latencies)
        else "sin datos",
        "tokens": fmt_tokens(
            sum(x for x in inputs if x is not None) if any(x is not None for x in inputs) else None,
            sum(x for x in outputs if x is not None)
            if any(x is not None for x in outputs)
            else None,
        ),
        "cost": fmt_cost(sum(costs), currency)
        if costs and all(c is not None for c in costs)
        else "sin datos",
    }

def build_table_rows(results: dict) -> str:
    rows = []
    currencies = {
        side: ((results["contenders"].get(side) or {}).get("price") or {}).get("currency")
        for side in SIDES
    }
    for task in results["tasks"]:
        t = {side: _side_task_totals(task["results"][side], currencies[side]) for side in SIDES}
        diff = f"<span>{e(task.get('difficulty'))}</span>" if task.get("difficulty") else ""
        cells = [f'<th scope="row" class="task">{e(task["title"])}{diff}</th>']
        for side in SIDES:
            status = (
                f'<span class="status">{e(t[side]["status"])}</span>' if t[side]["status"] else ""
            )
            cells.append(f'<td class="mono {side}">{e(t[side]["tests"])}{status}</td>')

```

```

        for key in ("time", "tokens", "cost"):
            for side in SIDES:
                cells.append(f"<td>{e(t[side][key])}</td>")
            rows.append("    <tr>" + "".join(cells) + "</tr>")
        return "\n".join(rows)

def build_details(results: dict) -> str:
    runs = results.get("runs", 1)
    blocks = []
    for task in results["tasks"]:
        summary_bits = []
        for side in SIDES:
            attempts = task["results"][side]
            passed = sum(a["passed"] for a in attempts)
            total = sum(a["total"] for a in attempts)
            summary_bits.append(f'<span class="{side}">{side.upper()} {passed}/{total}</span>')
        columns = []
        for side in SIDES:
            spec = results["contenders"][side]["spec"]
            attempts_html = [_attempt_html(attempt, runs) for attempt in task["results"][side]]
            columns.append(
                f'<div><div class="side-head {side}">{side.upper()} · {e(spec)}</div>'
                + "".join(attempts_html)
                + "</div>"
            )
        blocks.append(
            "<details>"
            f'<summary><span>{e(task["title"])}</span>'
            f'<span class="muted">{e(task["id"])}</span><span class="spacer"></span>'
            + " ".join(summary_bits)
            + "</summary>"
            '<div class="detail-body">'
            f'<div class="statement">{e(task.get("statement", ""))}</div>'
            f'<div class="duo">{" ".join(columns)}</div>'
            "</div></details>"
        )
    return "\n".join(blocks)

def _attempt_html(attempt: dict, runs: int) -> str:
    head = []
    if runs > 1:
        head.append(f"<strong>Ejecución {e(attempt.get('run', '?'))}</strong>")
    head.append(f"{e(attempt['passed'])}/{e(attempt['total'])} tests")
    if attempt["status"] == "ok":
        failing = attempt["failed"] + attempt["errors"]
        head.append("resuelta" if attempt["solved"] else plural(failing, "falla", "fallan"))
    else:
        head.append(e(STATUS_LABELS.get(attempt["status"], attempt["status"])))
    if attempt.get("latency_s") is not None:
        head.append(fmt_seconds(attempt["latency_s"]))
    parts = [f'<div class="attempt"><div class="attempt-head">{" ".join(head)}</div>']
    if attempt.get("message"):
        parts.append(f'<p class="msg">{e(attempt["message"])}</p>')
    if attempt.get("code"):
        parts.append(f'<pre><code class="language-python">{e(attempt["code"])}</code></pre>')
    elif attempt.get("response"):
        parts.append(f'<pre>{e(attempt["response"])}</pre>')
    if attempt.get("output"):
        parts.append(
            '<details class="out"><summary>Salida de pytest</summary>'
            f'<pre>{e(attempt["output"])}</pre></details>'
        )
    parts.append("</div>")
    return "".join(parts)

def build_warning(runs: int) -> str:
    if runs <= 1:
        return (
            "<strong>Una sola ejecución es una señal débil.</strong> "
            "Los modelos no son deterministas: la misma pregunta puede dar otra respuesta mañana. "
            "Repite el duelo con <code>--runs N</code> y con tareas de tu propio trabajo antes de "
            "sacar conclusiones."
        )
    return (

```

```

        "<strong>Pocas ejecuciones siguen siendo una señal débil.</strong> "
        f"Aquí hay {runs} ejecuciones por tarea, mejor que una, pero sigue siendo una muestra "
        "pequeña. Usa tareas de tu propio trabajo y mira el código, no solo el marcador."
    )

def _verdict(summary: dict, runs: int = 1) -> str:
    a, b = summary["a"]["tests_passed"], summary["b"]["tests_passed"]
    summed = f" (suma de {runs} ejecuciones)" if runs > 1 else ""
    if a == b:
        return "empate" + summed
    side = "a" if a > b else "b"
    return f'gana <strong class="{side}">{side.upper()}</strong> por {abs(a - b)}{summed}'

def _short_label(spec: str) -> str:
    kind, _, model = spec.partition(":")
    return model or kind

# ----- API

def render_report(results: dict) -> str:
    summary = results.get("summary") or summarize(results)
    spec_a = results["contenders"]["a"]["spec"]
    spec_b = results["contenders"]["b"]["spec"]
    runs = int(results.get("runs", 1))
    n_tasks = len(results["tasks"])
    fictitious = [
        side
        for side in SIDES
        if ((results["contenders"][side] or {}).get("price") or {}).get("fictitious")
    ]
    price_note = (
        "Precios ficticios de demostración, no reales."
        if fictitious
        else "Coste = entrada/1e6 × tarifa + salida/1e6 × tarifa · «sin datos» si no hay precio."
    )
    template = Template(
        resources.files("modelduel.report").joinpath("template.html").read_text("utf-8")
    )
    return template.substitute(
        version=e(results.get("version", __version__)),
        page_title=e(f"_{short_label(spec_a)} vs {short_label(spec_b)} · modelduel"),
        date_human=e(fmt_date(results.get("created_at", ""))),
        date_iso=e(results.get("created_at", "")),
        label_a=e(_short_label(spec_a)),
        label_b=e(_short_label(spec_b)),
        spec_a=e(spec_a),
        spec_b=e(spec_b),
        lede=e(
            f"{plural(n_tasks, 'tarea', 'tareas')} · "
            f"{plural(runs, 'ejecución', 'ejecuciones')} por tarea · "
            f"límite de {results.get('timeout_s', 20):g} s por ejecución de tests"
        ),
        score_a=summary["a"]["tests_passed"],
        score_b=summary["b"]["tests_passed"],
        score_total_a=summary["a"]["tests_total"],
        score_total_b=summary["b"]["tests_total"],
        verdict=_verdict(summary, runs),
        metrics=build_metrics(summary, runs),
        warning=build_warning(runs),
        chart=task_chart(results["tasks"]),
        table_caption=e(
            "Totales por tarea"
            + (f" sumando {runs} ejecuciones." if runs > 1 else ".")
            + " El tiempo es el que tarda el modelo en responder."
        ),
        table_rows=build_table_rows(results),
        details=build_details(results),
        price_note=e(price_note),
    )

def write_report(results: dict, out_dir: Path) -> Path:
    out_dir = Path(out_dir)

```

```
out_dir.mkdir(parents=True, exist_ok=True)
path = out_dir / "index.html"
path.write_text(render_report(results), encoding="utf-8")
return path
```

src/modelduel/report/template.html · 245 líneas

```
<!doctype html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="generator" content="modelduel $version">
<title>$page_title</title>
<link rel="icon" href="data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 16 16'%3E%3Crect
  width='16' height='16' fill='%230f0f11'/%3E%3Crect x='2' y='4' width='5' height='8' fill='%23b5e853'/%3E%3Crect
  x='9' y='4' width='5' height='8' fill='%23ff7eb6'/%3E%3C/svg%3E">
<style>
:root {
  --bg: #0f0f11;
  --panel: #151518;
  --text: #e8e8ea;
  --muted: #8d8d96;
  --line: #2a2a2f;
  --a: #b5e853;
  --b: #ff7eb6;
  --mono: ui-monospace, "JetBrains Mono", "SF Mono", "Cascadia Code", Menlo, Consolas, monospace;
  --sans: system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif;
}
* { box-sizing: border-box; }
html { background: var(--bg); color: var(--text); }
body {
  margin: 0; background: var(--bg); color: var(--text);
  font: 16px/1.55 var(--sans);
  -webkit-font-smoothing: antialiased;
}
.wrap { max-width: 1040px; margin: 0 auto; padding: 56px 24px 72px; }
a { color: inherit; text-decoration-color: var(--line); text-decoration-offset: 3px; }
a:hover { text-decoration-color: currentColor; }
.a { color: var(--a); }
.b { color: var(--b); }
.muted { color: var(--muted); }
.mono { font-family: var(--mono); }
.sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow: hidden; clip: rect(0 0 0 0); white-space: nowrap; border: 0; }
.eyebrow {
  font: 12px/1.4 var(--mono); letter-spacing: .12em; text-transform: uppercase;
  color: var(--muted); display: flex; flex-wrap: wrap; gap: 6px 18px;
}
h1 {
  font: 700 clamp(34px, 6.4vw, 68px)/1.02 var(--mono); letter-spacing: -.035em;
  margin: 22px 0 14px; overflow-wrap: anywhere;
}
h1.vs { color: var(--muted); font-weight: 400; font-size: .5em; letter-spacing: 0; vertical-align: .35em; }
h2 {
  font: 600 13px/1.4 var(--mono); letter-spacing: .14em; text-transform: uppercase;
  color: var(--muted); margin: 64px 0 18px; padding-bottom: 10px; border-bottom: 1px solid var(--line);
}
.lede { color: var(--muted); margin: 0; font-size: 15px; }

/* Marcador */
.board {
  margin-top: 44px; border-top: 1px solid var(--line); border-bottom: 1px solid var(--line);
  display: grid; grid-template-columns: 1fr auto 1fr; align-items: center; gap: 8px 24px;
  padding: 30px 0;
}
.side-name { font: 13px/1.3 var(--mono); color: var(--muted); overflow-wrap: anywhere; }
.side-name b { display: block; font-size: 12px; letter-spacing: .14em; margin-bottom: 4px; }
.board .left { text-align: left; }
.board .right { text-align: right; }
.big { font: 700 clamp(56px, 12vw, 128px)/.9 var(--mono); letter-spacing: -.05em; font-variant-numeric: tabular-nums; }
.big small { font-size: .28em; color: var(--muted); letter-spacing: 0; font-weight: 400; }
.colon { font: 300 clamp(40px, 8vw, 88px)/.9 var(--mono); color: var(--muted); text-align: center; }
.verdict { grid-column: 1 / -1; text-align: center; font: 14px/1.4 var(--mono); color: var(--muted); margin-top: 10px; }
.verdict strong { font-weight: 600; }

/* Métricas enfrentadas */
.metrics { margin: 8px 0 0; }
.metric {
  display: grid; grid-template-columns: 1fr minmax(200px, 1.3fr) 1fr; align-items: center;
  gap: 16px; padding: 16px 0; border-bottom: 1px solid var(--line);
}
```

```

}
.metric .val { font: 500 20px/1.2 var(--mono); font-variant-numeric: tabular-nums; }
.metric .val.b-side { text-align: right; }
.metric .val.lose .num { opacity: .8; }
.metric .val .mark { font-size: 9px; vertical-align: 5px; margin: 0 8px; }
.metric .val .sub { display: block; font: 12px/1.4 var(--mono); color: var(--muted); margin-top: 4px; }
.metric .mid { text-align: center; }
.metric .label { font: 12px/1.3 var(--mono); letter-spacing: .1em; text-transform: uppercase; color: var(--muted);
  margin-bottom: 8px; }
.metric svg { display: block; width: 100%; height: 6px; }
.hint { font: 11px/1.3 var(--mono); color: var(--muted); margin-top: 6px; }

/* Advertencia */
.warn {
  margin: 40px 0 0; padding: 18px 22px; border: 1px solid var(--line); border-left: 3px solid var(--text);
  background: var(--panel); font-size: 15px;
}
.warn strong { font-family: var(--mono); font-weight: 600; }
.warn code { font-family: var(--mono); font-size: .92em; }

/* Gráfica por tarea */
.bar-row { padding: 14px 0 16px; border-bottom: 1px solid var(--line); }
.bar-title { font: 600 14px/1.4 var(--mono); margin-bottom: 10px; }
.bar-title span { font-weight: 400; color: var(--muted); font-size: 12px; margin-left: 10px; }
.bar-line { display: grid; grid-template-columns: 1fr 64px; align-items: center; gap: 14px; margin-top: 6px; }
.bar-line svg { display: block; width: 100%; height: 6px; }
.bar-line span { font: 12px/1 var(--mono); text-align: right; font-variant-numeric: tabular-nums; }
.track { fill: var(--line); }
.fill-a { fill: var(--a); }
.fill-b { fill: var(--b); }
.txt-a { fill: var(--a); }
.txt-b { fill: var(--b); }
.legend { display: flex; gap: 22px; font: 12px var(--mono); color: var(--muted); margin: 0 0 14px; }
.legend i { display: inline-block; width: 18px; height: 3px; vertical-align: middle; margin-right: 8px; }
.legend .la { background: var(--a); }
.legend .lb { background: var(--b); }

/* Tabla */
.table-wrap { overflow-x: auto; }
table { width: 100%; border-collapse: collapse; font-size: 14px; font-variant-numeric: tabular-nums; }
caption { text-align: left; color: var(--muted); font-size: 13px; padding-bottom: 10px; caption-side: bottom; padding-top: 10px; }
th, td { padding: 11px 9px; border-bottom: 1px solid var(--line); text-align: right; white-space: nowrap; }
th { font: 11px/1.3 var(--mono); letter-spacing: .1em; text-transform: uppercase; color: var(--muted); font-weight: 500; }
thead tr:first-child th { border-bottom: 0; padding-bottom: 2px; }
thead tr:first-child th:first-child, tbody th:first-child { text-align: left; white-space: normal; padding-left: 0; min-width: 170px; }
thead tr + tr th { padding-top: 0; }
th.task { font: 14px/1.4 var(--mono); letter-spacing: 0; text-transform: none; color: var(--text); }
th.task span { display: block; font-size: 12px; color: var(--muted); font-family: var(--sans); }
td.mono { font-family: var(--mono); }
.status { font: 11px var(--mono); color: var(--muted); display: block; }

/* Detalle */
details { border-bottom: 1px solid var(--line); }
details > summary {
  cursor: pointer; list-style: none; padding: 16px 0; display: flex; gap: 16px; align-items: baseline;
  font: 15px/1.4 var(--mono);
}
details > summary::-webkit-details-marker { display: none; }
details > summary::before { content: "+"; color: var(--muted); width: 14px; flex: none; }
details[open] > summary::before { content: "-"; }
details > summary .spacer { flex: 1; }
details > summary:focus-visible { outline: 2px solid var(--text); outline-offset: 4px; }
.detail-body { padding: 0 0 28px 30px; }
.statement { white-space: pre-wrap; font-size: 14px; color: var(--muted); border-left: 1px solid var(--line); padding: 2px 0 2px 16px; margin: 0 0 22px; max-height: 320px; overflow: auto; }
.duo { display: grid; grid-template-columns: 1fr 1fr; gap: 24px; }
.duo > div { min-width: 0; }
.side-head { font: 600 12px/1.4 var(--mono); letter-spacing: .12em; text-transform: uppercase; padding-bottom: 8px; border-bottom: 2px solid var(--text); margin-bottom: 12px; overflow-wrap: anywhere; }
.attempt + .attempt { margin-top: 22px; }
.attempt-head { font: 12px/1.4 var(--mono); color: var(--muted); margin-bottom: 8px; }
.attempt-head strong { color: var(--text); font-weight: 600; }
pre {
  margin: 0; padding: 14px 16px; background: var(--panel); border: 1px solid var(--line);

```

```

    font: 12.5px/1.55 var(--mono); overflow: auto; max-height: 460px; tab-size: 4; color: var(--text);
  }
  details.out { border: 0; margin-top: 8px; }
  details.out > summary { padding: 6px 0; font-size: 12px; color: var(--muted); }
  .msg { font-size: 13px; color: var(--muted); margin: 0 0 8px; }

  footer { margin-top: 72px; padding-top: 18px; border-top: 1px solid var(--line); font: 12px/1.7 var(--mono); color:
    var(--muted); display: flex; flex-wrap: wrap; gap: 4px 22px; }

  @media (max-width: 900px) {
    .duo { grid-template-columns: 1fr; }
  }

  @media (max-width: 720px) {
    .wrap { padding: 36px 16px 56px; }
    .metric { grid-template-columns: 1fr 1fr; }
    .metric .mid { grid-column: 1 / -1; order: -1; }
    .duo { grid-template-columns: 1fr; }
    .detail-body { padding-left: 0; }
  }

  @media print {
    :root { --bg: #fff; --panel: #f6f6f6; --text: #111; --muted: #555; --line: #d4d4d8; --a: #4d7c0f; --b: #be185d; }
    html, body { background: #fff; color: #111; }
    .wrap { max-width: none; padding: 0; }
    h1 { font-size: 34px; }
    .big { font-size: 64px; }
    h2 { margin-top: 32px; }
    pre { max-height: none; white-space: pre-wrap; overflow: visible; }
    .statement { max-height: none; }
    details > summary::before { content: ""; width: 0; }
    details::details-content { content-visibility: visible; display: block; }
    .bar-line svg, .metric svg { print-color-adjust: exact; -webkit-print-color-adjust: exact; }
    .table-wrap { overflow: visible; }
    table { font-size: 11px; }
    th, td { white-space: normal; padding: 6px 4px; }
    section, .metric, details { break-inside: avoid; }
    a { text-decoration: none; }
  }
</style>
</head>
<body>
<div class="wrap">
<header>
  <div class="eyebrow"><span>modelduel</span><span>Informe de duelo</span><span>${date_human}</span><span>v${version}</span></div>
  <h1><span class="a">${label_a}</span> <span class="vs">vs</span> <span class="b">${label_b}</span></h1>
  <p class="lede">${lede}</p>
</header>

<main>
<section class="board" aria-label="Marcador">
  <div class="left">
    <div class="side-name"><b class="a">A</b>${spec_a}</div>
    <div class="big a">${score_a}<small>/${score_total_a}</small></div>
  </div>
  <div class="colon" aria-hidden="true"></div>
  <div class="right">
    <div class="side-name"><b class="b">B</b>${spec_b}</div>
    <div class="big b">${score_b}<small>/${score_total_b}</small></div>
  </div>
  <p class="verdict">tests superados · ${verdict}</p>
</section>

<section class="metrics" aria-label="Métricas enfrentadas">
${metrics}
</section>

<aside class="warn" role="note">
  ${warning}
</aside>

<h2>Tests superados por tarea</h2>
<div class="chart">
  <div class="legend" aria-hidden="true"><span><i class="la"></i>A · ${spec_a}</span><span><i class="lb"></i>B ·
    ${spec_b}</span></div>
  ${chart}
</div>

```

```

</div>

<h2>Tabla por tarea</h2>
<div class="table-wrap">
<table>
  <caption>${table_caption}</caption>
  <thead>
    <tr><th scope="col" rowspan="2">Tarea</th><th scope="colgroup" colspan="2">Tests</th><th scope="colgroup"
colspan="2">Tiempo del modelo</th><th scope="colgroup" colspan="2">Tokens entrada / salida</th><th scope="colgroup"
colspan="2">Coste</th></tr>
    <tr><th scope="col" class="a">A</th><th scope="col" class="b">B</th><th scope="col" class="a">A</th><th
scope="col" class="b">B</th><th scope="col" class="a">A</th><th scope="col" class="b">B</th><th scope="col"
class="a">A</th><th scope="col" class="b">B</th></tr>
  </thead>
  <tbody>
${table_rows}
  </tbody>
</table>
</div>

<h2>Código y salida de los tests</h2>
${details}
</main>

<footer>
  <span>Generado por modelduel ${version} · ${date_iso}</span>
  <span>${price_note}</span>
  <span><a href="https://github.com/BertMarti/modelduel">github.com/BertMarti/modelduel</a></span>
</footer>
</div>
</body>
</html>

```

examples/tasks/slugify/task.md · 25 líneas

Convertir un texto en slug

Escribe una función de Python con esta firma exacta:

```

```python
def slugify(text: str, separator: str = "-") -> str:
```

```

Debe convertir cualquier texto en un *slug* apto para una URL siguiendo estas reglas:

1. El resultado va en minúsculas.
2. Las letras con tilde, diéresis u otros diacríticos se convierten en su letra base (`á` → `a`, `ñ` → `n`, `ü` → `u`, `ç` → `c`).
3. Solo se conservan letras ASCII (`a`-`z`) y dígitos (`0`-`9`).
4. Cualquier secuencia de uno o más caracteres no conservados (espacios, signos de puntuación, barras, guiones, puntos...) se sustituye por **un único** `separator`.
5. El resultado nunca empieza ni termina con el separador.
6. Si no queda ningún carácter conservado, devuelve la cadena vacía `""`.

Ejemplos:

```

- `slugify("Canción de Añoranza")` → `"cancion-de-anoranza"`
- `slugify("Python 3.12 en 2026")` → `"python-3-12-en-2026"`
- `slugify("Árbol Genealógico", separator="_")` → `"arbol_genealogico"`

```

examples/tasks/slugify/test_task.py · 39 líneas

```
from solution import slugify

def test_basico():
    assert slugify("Hola Mundo") == "hola-mundo"

def test_acentos_y_enie():
    assert slugify("Canción de Añoranza") == "cancion-de-anoranza"
    assert slugify("Pingüino FRANÇAIS") == "pinguino-francais"

def test_colapsa_signos_y_espacios():
    assert slugify("¡¡Hola, mundo!! ¿Qué tal?") == "hola-mundo-que-tal"

def test_sin_separadores_en_los_extremos():
    assert slugify("--Ya está-- ") == "ya-esta"

def test_punto_entre_digitos_separa():
    assert slugify("Python 3.12 en 2026") == "python-3-12-en-2026"

def test_barra_y_guion_bajo_separan():
    assert slugify("Entrada/Salida") == "entrada-salida"
    assert slugify("nombre_de_variable") == "nombre-de-variable"

def test_sin_caracteres_validos_devuelve_vacio():
    assert slugify("") == ""
    assert slugify(" ") == ""
    assert slugify("¡;!?") == ""

def test_separador_personalizado():
    assert slugify("Árbol Genealógico", separator="_") == "arbol_genealogico"
    assert slugify(" a · b ", separator=".") == "a.b"
```

.github/workflows/ci.yml · 56 líneas

```
name: CI

on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  test:
    name: Lint, tests y demo (${{ matrix.os }})
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: pyproject.toml

      - name: Instalar
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev]"

      - name: Lint
        run: ruff check .

      - name: Formateo
        run: ruff format --check .

      - name: Tests
        run: pytest --cov --cov-report=term

      - name: Demo con respuestas grabadas
        run: modelduel run examples/tasks --a replay:alfa --b replay:beta --out site/demo

      - name: Comprobar el informe
        shell: bash
        run: |
          test -s site/demo/index.html
          test -s site/demo/results.json
          grep -q "Una sola ejecución es una señal débil" site/demo/index.html
          if grep -qi "<script" site/demo/index.html; then echo "El informe no debe tener JavaScript"; exit 1; fi
```

`.github/workflows/deploy.yml` · 53 líneas

```
name: Deploy

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
          cache-dependency-path: pyproject.toml

      - name: Instalar
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev]"

      - name: Generar el informe de demostración
        run: modelduel run examples/tasks --a replay:alfa --b replay:beta --out site/demo

      - uses: actions/configure-pages@v5

      - uses: actions/upload-pages-artifact@v3
        with:
          path: site

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - name: Desplegar en GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```